

Riassunto Ingegneria del Software

ANNO 2025/2026

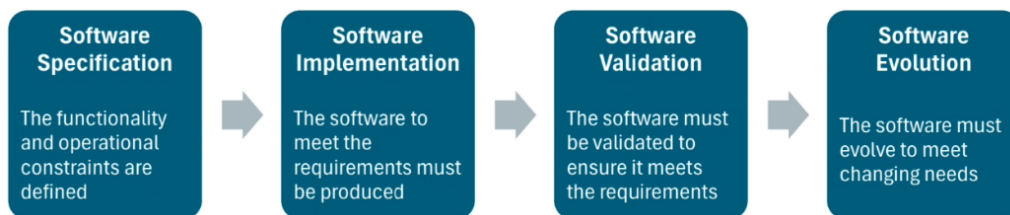
Marta Pagliuso

**Professor
Tramontana - Breve**

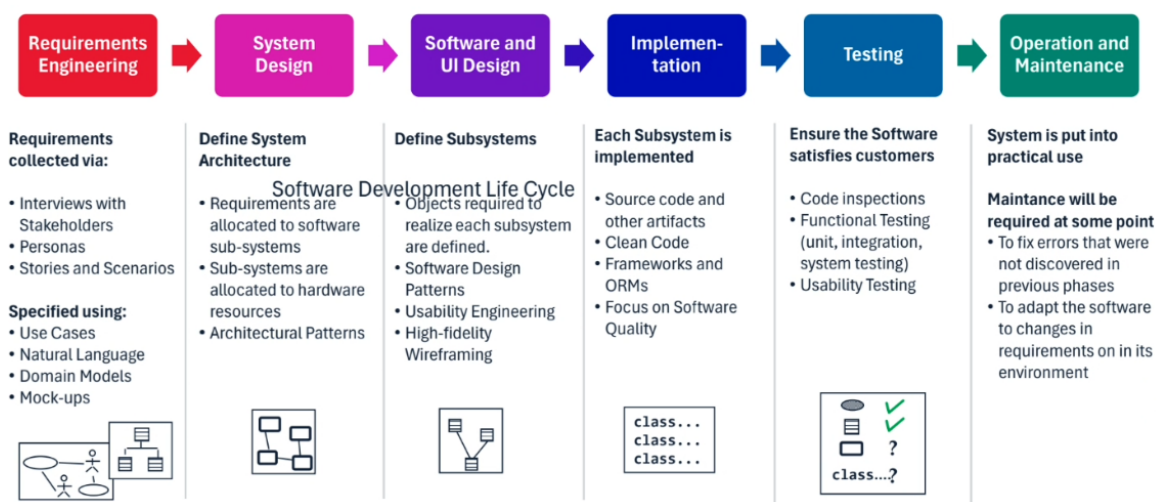
CAPITOLO 1

Processi Software

Un **processo software** è un'insieme di attività correlate che portano alla produzione di un sistema software. Non esiste un processo universale che funzioni sempre, e ne esistono e vengono utilizzati molti diversi. Tutti includono, in qualche forma, le seguenti attività fondamentali:



Un **modello di processo software** o **ciclo di vita dello sviluppo software (SDLC)** è una rappresentazione semplificata di un processo software. Il primo modello mostrato è quello a *cascata*:



1.1 REQUISITI

L'obiettivo principale è capire **cosa** il Software dovrebbe fare, e non *come* implementarlo. In particolare abbiamo:

- Analisi accurata delle esigenze dell'utente e del dominio del problema;
- Coinvolgimento di clienti, utenti finali e ingegneri del Software;
- Il risultato è un documento di specifiche di requisiti software.

1.2 SYSTEM AND SOFTWARE/UI DESIGN

L'obiettivo principale è progettare una struttura (**architettura**) adeguata per il software. Ci sono due diversi livelli:

- **System Design:** architettura generale del sistema
 - Decomposizione in moduli e componenti;
 - Assegnazioni di funzionalità ai moduli e i moduli ai componenti hardware;
 - Definizione di relazioni e collaborazioni tra moduli.
- **Software and UI design:** dettagli su come implementare i moduli
 - Include la progettazione dell'architettura software di basso livello (esempio: di quali classi abbiamo bisogno?);
 - Include la prototipazione dell'interfaccia utente.

L'output è un insieme di specifiche spesso rappresentate con linguaggi come **UML**.

1.3 IMPLEMENTAZIONE

L'obiettivo principale è "**tradurre**" (in alta qualità) le specifiche di progettazione in un linguaggio/tecnologia di programmazione a scelta.

1.3.1 Testing

- **Verification:** il sistema è conforme alle specifiche?
 - Questo in genere viene effettuato tramite il test del programma, eseguendolo in un ambiente controllato e verificando che il suo comportamento sia corretto rispetto alle specifiche.
- **Validation:** il sistema soddisfa le aspettative dei clienti?
 - Questo viene generalmente fatto definendo ed eseguendo test di accettazione.

1.4 OPERAZIONE AND MANUTENZIONE

- **Operazione:** il sistema viene distribuito e installato per i clienti, e viene messo in uso effettivo.
- **Manutenzione:** il software subirà delle modifiche nel tempo.
 - Le esigenze del cliente potrebbero cambiare;
 - Potrebbero emergere bug sfuggiti in fase di verifica e validazione.

CAPITOLO 2

Qualità Software

Non esiste una sola "qualità", ma molti approcci e punti di vista diversi.

Nello standard, il concetto di qualità del software è modellato da:

- **Modello di qualità del prodotto:** composto da 9 caratteristiche (ulteriormente divise in sottocaratteristiche) che si riferiscono alle proprietà qualitative dei prodotti.
- **Modello di qualità in uso:** composto da 3 caratteristiche (ulteriormente divise in sottocaratteristiche) che possono influenzare le parti interessate quando i prodotti o i sistemi vengono utilizzati in uno specifico contesto d'uso.

2.1 ISO 25002 MODELLO DI QUALITÀ



2.1.1 Idoneità

L'**idoneità funzionale** risponde alla domanda: *"Questo software fa quello che deve fare per risolvere il problema dell'utente?"*. Per valutare se un software è "idoneo", la norma divide questa caratteristica in tre aspetti:

- **Completezza funzionale:** Ci sono tutte le funzioni che servono?
- **Correttezza funzionale:** Le funzioni danno il risultato giusto?
- **Appropriatezza funzionale:** Le funzioni sono pensate in modo intelligente per quel compito? Riguarda quanto una funzione sia sensata e adatta per aiutare l'utente a raggiungere il suo scopo senza sforzi inutili o passaggi illogici.

2.1.2 Affidabilità

L'**affidabilità** risponde alla domanda: *"Questo software mi lascerà a piedi sul più bello?"*.

Questa caratteristica si divide in:

- **Faultlessness (Assenza di guasti)**: riguarda la capacità di funzionare senza generare errori durante le normali operazioni.
- **Availability (Disponibilità)**: riguarda il famoso "uptime", ovvero la percentuale di tempo in cui il servizio è effettivamente accessibile agli utenti.
- **Fault tolerance (Tolleranza ai guasti)**: riguarda la capacità del software di non andare nel panico se un componente hardware o un altro software collegato smette di funzionare.
- **Recoverability (Recuperabilità)**: riguarda la capacità di far ripartire il sistema e salvare il salvabile dopo un blocco totale o un'interruzione di corrente.

2.1.3 Efficienza

L'**efficienza** risponde a questa domanda: *"Quante risorse consuma il software per fare il suo lavoro?"*. Non basta che un programma funzioni; deve farlo senza prosciugare la batteria del telefono, senza saturare la RAM e senza farti aspettare mezz'ora.

Questa caratteristica si divide in:

- **Time behaviour (Comportamento temporale)**: riguarda i tempi di risposta (quanto aspetti dopo un clic) e il *throughput* (quante operazioni riesce a gestire in un secondo).
- **Resource utilization (Utilizzo delle risorse)**: un software può essere velocissimo, ma se per esserlo usa il 100% della CPU o tutta la memoria disponibile, non è efficiente.
- **Capacity (Capacità)**: definisce i limiti estremi che il sistema può reggere prima di degradare o smettere di funzionare.

2.1.4 Usabilità

L'**usabilità** risponde alla domanda: *"L'utente riesce a usare il software in modo facile, piacevole e senza frustrazioni?"*.

Questa caratteristica si divide in:

- **Recognizability (Riconoscibilità)**: è la capacità del software di far capire all'utente, al primo sguardo, se è lo strumento giusto per i suoi bisogni.
- **Learnability (Apprendibilità)**: riguarda lo sforzo che l'utente deve fare per diventare capace di usare le funzioni del sistema.
- **Operability (Operabilità)**: si riferisce alla facilità con cui l'utente può effettivamente interagire con l'interfaccia (menu logici, bottoni grandi quanto serve, navigazione fluida).
- **User error protection (Protezione dagli errori)**: riguarda la capacità del software di evitare che l'utente compia errori o di rendere facile rimediare se li compie.
- **Inclusivity (Inclusività)**: il software deve essere accessibile a persone con disabilità (es. compatibile con lettori di schermo per non vedenti), ma anche a persone di culture diverse o con diverse capacità tecniche.

2.1.5 Sicurezza

La **sicurezza** risponde alla domanda: *"Il sistema protegge i miei dati dai malintenzionati e si assicura che solo chi ne ha il diritto possa farci qualcosa?"*.

Questa caratteristica si divide in:

- **Confidentiality (Riservatezza):** riguarda il fatto che le informazioni devono essere lette solo da chi ha l'autorizzazione per farlo.
- **Integrity (Integrità):** garantisce che i dati non vengano modificati o cancellati da chi non è autorizzato (o per via di un errore del sistema).
- **Non-repudiation (Non ripudio):** serve ad avere le "prove schiaccianti" che un'azione è avvenuta, in modo che chi l'ha compiuta non possa dire "Non sono stato io!".
- **Accountability (Responsabilità/Tracciabilità):** riguarda la capacità del sistema di registrare tutte le azioni e collegarle in modo univoco a chi le ha eseguite (di solito tramite i "file di log").
- **Authenticity (Autenticità):** riguarda la verifica dell'identità dell'utente (o di una risorsa esterna) prima di farlo entrare.

2.1.6 Compatibilità

La **compatibilità** risponde alla domanda: *"Questo software sa stare al mondo e giocare in squadra con gli altri programmi?"*.

Questa caratteristica si divide in:

- **Co-existence (Coesistenza):** più software installati sullo stesso computer o smartphone devono condividere la stessa memoria (RAM), lo stesso processore e la stessa rete. Un software ha una buona coesistenza se fa il suo lavoro senza "rubare" tutte le risorse, danneggiando o rallentando gli altri programmi.
- **Interoperability (Interoperabilità):** qui non si tratta di ignorarsi pacificamente, ma di collaborare attivamente. È la capacità di due sistemi distinti di passarsi dei dati e di saperli leggere e utilizzare correttamente.

2.1.7 Manutenibilità

La **manutenibilità** risponde alla domanda: *"Quanto è facile, veloce e sicuro mettere le mani nel codice per cambiarlo?"*.

Questa caratteristica si divide in:

- **Modularity (Modularità):** riguarda l'organizzazione del codice in blocchi (moduli) indipendenti. Un cambiamento in un modulo non deve scatenare un effetto domino sugli altri.
- **Reusability (Riutilizzabilità):** è la capacità di prendere un pezzo di codice già scritto e usarlo come base solida per costruire un software completamente diverso, ottimizzando tempi e costi.
- **Modifiability (Modificabilità):** riguarda lo sforzo necessario per cambiare il software senza introdurre nuovi difetti.
- **Testability (Testabilità):** riguarda la facilità con cui gli sviluppatori possono creare ed eseguire dei test per stabilire se le modifiche appena fatte rispettano i criteri richiesti, senza aver rotto null'altro.

2.1.8 Flessibilità

La **flessibilità** risponde alla domanda: *"Questo programma è pronto ad affrontare il cambiamento, o è rigido e morirà al primo imprevisto?"*.

Questa caratteristica si divide in:

- **Adaptability (Adattabilità):** è la capacità del software di funzionare su hardware, sistemi operativi o contesti operativi diversi da quelli per cui era stato originariamente pensato, senza dover riscrivere tutto il codice da zero.
- **Scalability (Scalabilità):** riguarda la capacità del sistema di gestire un aumento (o una diminuzione) improvviso di dati o di utenti, semplicemente aggiungendo o rimuovendo risorse hardware, senza andare in blocco.
- **Installability (Installabilità):** riguarda la semplicità, l'efficacia e la velocità con cui un software può essere installato e, fattore spesso ignorato, disinstallato. Un programma di qualità non deve lasciare "spazzatura" nel computer quando decidi di rimuoverlo.
- **Replaceability (Sostituibilità):** è la capacità di un nuovo prodotto di prendere il posto di un software più vecchio, svolgendo lo stesso identico scopo nello stesso ambiente, senza far impazzire gli utenti o rompere le dinamiche aziendali.

2.1.9 Safety

La **safety** risponde alla domanda: *"Se questo programma impazzisce o si guasta, qualcuno rischia di farsi male, di perdere capitali o di causare un disastro ambientale?"*.

Questa caratteristica si divide in:

- **Fail safe (A prova di guasto):** riguarda la capacità del sistema di mettersi automaticamente in una condizione in cui non può nuocere a nessuno in caso di errore critico.
- **Risk identification (Identificazione del rischio):** è la capacità del software di monitorare la situazione esterna e accorgersi se una serie di eventi sta portando a una situazione di pericolo inaccettabile.
- **Hazard warning (Avviso di pericolo):** non basta capire che c'è un rischio; bisogna avvisare chi di dovere in modo che possa reagire prima che il danno diventi reale.

2.2 MODELLI DI QUALITÀ IN USO



2.2.1 Usabilità

L'**usabilità** in questo caso si concentra sull'utente. Risponde alla domanda: *"Il software è stato rilasciato nel mondo reale. Come se la cava l'essere umano che lo sta impugnando?"*.

Questa caratteristica si divide in:

- **Effectiveness (Efficacia):** è un parametro che misura in modo oggettivo l'accuratezza e la completezza con cui l'utente raggiunge il suo scopo. Non conta come ci arriva, conta se ci arriva.
- **Efficiency (Efficienza):** qui si misurano le risorse "spese" dall'utente (minuti persi, numero di clic, sforzo mentale e visivo) rispetto al risultato ottenuto.

- **Satisfaction (Soddisfazione):** misura se l'utente si è sentito a suo agio, se si è fidato del software, se lo ha trovato piacevole o se ha provato frustrazione.

2.2.2 Safety

La **safety** risponde alla domanda: *"Quando un essere umano usa questo software nella vita di tutti i giorni, che disastri rischia di causare a sé stesso, al portafogli o al mondo che lo circonda?"*.

Questa caratteristica si divide in:

- **Commercial Damage (Danno Commerciale / Rischio Economico):** riguarda la capacità del sistema di evitare perdite finanziarie o danni al business di chi lo sta usando.
- **Operator Health and Safety (Salute e sicurezza dell'operatore):** riguarda la protezione di chi "preme i tasti" o indossa il dispositivo. Rientrano qui non solo i pericoli fisici mortali, ma anche lo stress, la postura, l'affaticamento visivo o psicologico.
- **Public Health and Safety (Salute e sicurezza pubblica):** questo è il rischio che l'operatore del software fa correre alla collettività.
- **Environmental Harm (Danno Ambientale):** valuta se le decisioni prese usando il sistema portano a inquinamento, spreco di risorse o danni ecologici.

2.2.3 Flessibilità

La **flessibilità** risponde alla domanda: *"Il sistema si adatta fluidamente alle situazioni della vita reale, o costringe l'utente a fare salti mortali per usarlo?"*.

Questa caratteristica si divide in:

- **Context Conformity (Conformità al contesto):** riguarda la capacità del sistema di essere efficace, efficiente e sicuro nei contesti specifici per cui è stato originariamente pensato e progettato.
- **Context Extensibility (Estensibilità del contesto):** riguarda la capacità del software di adattarsi a situazioni, dispositivi o compiti diversi e imprevisi rispetto a quelli originali, senza dover fare pesanti modifiche al sistema.
- **Accessibility (Accessibilità):** valuta se, nel mondo reale, la persona (con disabilità o altri problemi) riesce effettivamente a "comprare" il software. (ad es. se una persona non vedente riesce effettivamente a comprare un biglietto per il treno).

CAPITOLO 3

Requisiti Software

I **requisiti** sono descrizioni di *cosa* il sistema dovrebbe fare:

- I servizi che il sistema deve fornire agli utenti;
- Vincoli operativi.

In alcuni casi, un requisito è una descrizione astratta e di alto livello di un servizio che il sistema dovrebbe fornire o un vincolo del sistema. Questi vengono chiamati **Requisiti Utente** (User Requirements), il cui focus è la prospettiva utente.

In altri casi, è una definizione più dettagliata e formale di un sistema di funzioni. Questi prendono il nome di **Requisiti di Sistema** (System Requirements), il cui focus è sul sistema da costruire.

3.1 REQUIREMENTS ENGINEERING (RE)

RE è una sotto-area dell'Ingegneria del Software che si occupa del processo di definizione dei requisiti per un software da sviluppare.

I requisiti sono spesso classificati come:

- **Requisiti Funzionali:**
 - Servizi che il sistema dovrebbe fornire;
 - Come il sistema dovrebbe reagire a particolari input o comportarsi in una data situazione.
- **Requisiti Non-Funzionali:**
 - Vincoli sui servizi o sulle funzioni offerti dal sistema;
 - Includono vincoli temporali, vincoli di processo o vincoli imposti dagli standard;
 - Spesso si applicano all'intero sistema piuttosto che alle singole funzionalità o servizi.

3.1.1 Requisiti Funzionali

Quando sono espressi come **requisiti funzionali dell'utente**, possono essere scritti in linguaggio naturale, in modo che possano essere compresi da persone non tecniche (ad esempio, utenti e manager).

Quando sono espressi come **requisiti funzionali del sistema**, dovrebbero essere dettagliati, descrivere dettagliatamente gli input e gli output del sistema e le eccezioni, in modo che gli ingegneri del software sappiano esattamente cosa implementare.

3.1.2 Requisiti Non-Funzionali

In genere, si specificano o si vincolano le caratteristiche del sistema nel suo complesso:

- Vincoli su come deve essere implementato (ad es: deve utilizzare il linguaggio Java);
- Specificano altre proprietà (ad es: tempi di risposta, utilizzo della memoria,...);

Spesso i requisiti non-funzionali sono più critici di quelli funzionali, in quanto, ad esempio, gli utenti potrebbero trovare una soluzione alternativa a un'implementazione non ottimale di un requisito funzionale, ma il mancato rispetto di un requisito non funzionale potrebbe rendere il sistema inutilizzabile.

I requisiti non funzionali possono derivare da:

- Caratteristiche richieste dal prodotto (**Requisiti di prodotto**): velocità di sistema, quantità di memoria, etc...;
- Le organizzazioni del cliente e degli sviluppatori (**Requisiti organizzativi**): linguaggio di programmazione richiesto, requisiti che specificano l'ambiente operativo, etc...;
- Fonti esterne (**Requisiti esterni**): requisiti legali, requisiti etici, etc...

3.2 REQUISITI DI DOMINIO

I requisiti dovrebbero essere:

1. **Chiari e facili da comprendere** (soprattutto i Requisiti Utente);
2. **Non ambigui**;
3. **Completi**;
4. **Consistenti** (ad esempio, non dovrebbero andare in conflitto tra di loro);
5. **Testabili**;
6. **Tracciabili**.

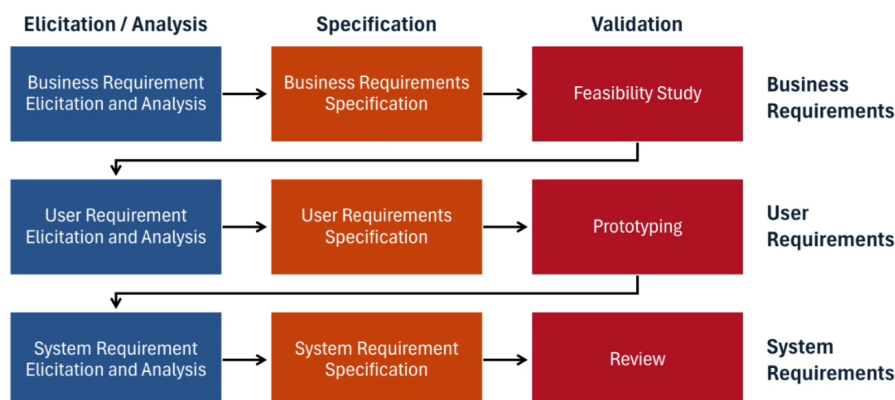
3.3 PROCESSO DI INGEGNERIA DEI REQUISITI

Tre step chiave:

1. **Analisi e raccolta dei requisiti**: scoprire i requisiti con le parti interessate;
2. **Specificazione dei requisiti**: conversione dei requisiti in un formato standardizzato;
3. **Validazione dei requisiti**: verifica che i requisiti definiscono effettivamente il sistema desiderato dal cliente.

In pratica, il processo di ingegneria dei requisiti non è lineare. Le sue attività sono infatti intercalate in un **processo iterativo** a diversi livelli di granularità.

- Nella prima iterazione, ci concentriamo sui requisiti a livello aziendale (Perché è necessario il progetto? Chi ne tratterà beneficio? Quando e dove avrà luogo?);
- Successivamente, definiamo i requisiti utente;
- Infine, definiamo i requisiti di sistema.



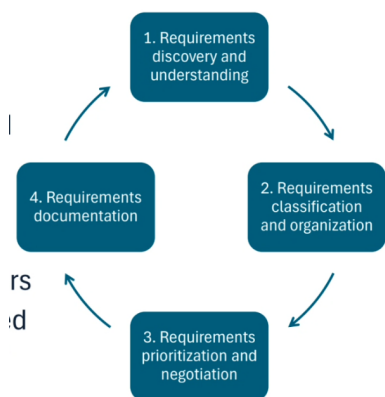
3.4 RACCOLTA E ANALISI DEI REQUISITI

La raccolta dei requisiti è la parte più cruciale della RE. Include la collaborazione tra diversi **stakeholders**. Sono persone che sono in qualche modo coinvolte nello sviluppo software del progetto, ad esempio:

- Persone che usano l'output del software;
- Gestione clienti e dirigenti;

Gli stakeholders non sanno cosa vogliono dal software (nel senso generale del termine). Non conoscono cosa è fattibile e cosa no, per cui potrebbero fare proposte irrealistiche. Questo porta a un possibile cambio dei requisiti durante questo processo.

Le fasi sono:



1. **Scoprire e comprendere:** interagire con gli stakeholders per scoprire i requisiti;
2. **Classificazione e organizzazione:** requisiti affini vengono raggruppati e organizzati;
3. **Definizione delle priorità e negoziazione:** risolvere conflitti dei requisiti nati da differenti bisogni degli stakeholders;
4. **Documentazione:** tenere traccia della prossima iterazione del processo di raccolta dei requisiti.

Esistono diversi approcci che possono essere utilizzati per scoprire i requisiti:

- Interviste con gli stakeholders;
- Osservazione e studi etnografici;
- Definizione delle Personas;
- Definizione degli Scenarios e le User Stories;
- Definizione dei mock-ups e i prototipi.

3.4.1 Interviste con gli stakeholders

Durante le interviste, il team RE pone agli stakeholders domande riguardo il loro lavoro, il sistema correntemente utilizzato e il sistema che deve essere sviluppato. I requisiti derivano dalle risposte a queste domande.

Le interviste possono essere:

- **Chiuse:** stakeholders rispondono a una lista predefinita di domande;
- **Aperte:** non c'è un'agenda predefinita.

Nella pratica però, le interviste sono di solito una combinazione di entrambi gli approcci.

Le sfide che si possono incontrare:

- Stakeholders usano linguaggio non colloquiale (gergo) e potrebbero dare dei requisiti per scontato;
- Stakeholders potrebbero avere una visione distorta e parziale riguardo il lavoro degli altri colleghi in differenti aree o dipartimenti;
- Stakeholders potrebbero essere riluttanti nel discutere requisiti e vincoli organizzativi, dovuti a dinamiche di potere all'interno dell'organizzazione.

3.4.2 Etnografia

L'**etnografia** è una tecnica di osservazione che può essere utilizzata per comprendere i processi operativi. In particolare è efficace nello scoprire:

- Requisiti che derivano da come le persone *effettivamente* lavorano;
- Requisiti che derivano dalla cooperazione e conoscenza delle attività di altre persone.

3.4.3 Storie

Le **storie** sono scritte come un testo narrativo e presentano una descrizione di alto livello del sistema usato.

3.4.4 Mockups a bassa fedeltà

Sono degli sketches dell'interfaccia utente del sistema. È un modo veloce ed efficace di visualizzare la struttura e il flusso di base del sistema. Vengono disegnate solo le funzionalità, quindi non c'è bisogno di concentrarsi sull'estetica.

I benefici sono:

- Facilitare la comprensione;
- Aiutare a scoprire nuovi requisiti;

- Coinvolgimento e feedback iterativo;
- Fondamento per lo sviluppo.

Si possono usare come tools carta e penna, oppure software digitali come Figma, draw.io ecc.

3.5 SPECIFICHE DEI REQUISITI

Di base, i requisiti dovrebbero dichiarare cosa il sistema dovrebbe fare e il design dovrebbe descrivere come lo fa. Dunque in pratica, i **requisiti** e il **design** sono inseparabili.

Sono possibili diversi approcci:

- **Linguaggio naturale:** i requisiti sono espressi come frasi numerate nel linguaggio naturale. Ogni frase dovrebbe esprimere un requisito;
- **Linguaggio naturale strutturato:** usa un form o un template standardizzato;
- **Modelli e notazioni semi-formali:** Diagrammi UML Use Case e altri modelli di dominio;
- **Specificazione formale:** queste notazioni sono basate su concetti matematici, come ad esempio macchine a stati finiti e infiniti, logiche temporali.

Ovviamente, ogni approccio viene usato in domini differenti.

3.5.1 Specifiche in linguaggio naturale (NL)

Usato per scrivere requisiti in quanto è **espressiva**, **intuitiva** e **universale**. Può esprimere sia i requisiti funzionali sia quelli non funzionali.

Come linee guida bisogna tenere in mente:

- Definire uno standard di formato per tutti i requisiti
 - Esempio: `{part of the system}_i shall {requirement}_i`
- Usare il linguaggio in modo coerente;
- Usare testo evidenziato per mettere in mostra le parti principali dei requisiti;
- Evitare l'uso di gergo tecnico;
- Includere una spiegazione del perché un requisito è necessario.

Presenta però anche dei problemi:

- Mancanza di chiarezza;
- Confusione tra i requisiti;
- Fusione dei requisiti (diversi requisiti potrebbero essere espressi insieme).

CAPITOLO 4

Diagrammi Use Case

Gli Use Cases sono un modo per descrivere le interazioni tra gli utenti e un sistema usando un modello grafico e un testo strutturato in linguaggio naturale. Sono una parte fondamentale degli UML.

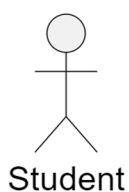
Si identificano:

- **Attori (Actors):** categoria di utenti (non necessariamente umani) di un sistema;
- **Use Cases:** tipi di interazioni (o features) offerte dal sistema.

Non vanno a definire *come* il sistema è implementato, ma *cosa* dovrebbe fare. Sono spesso documentati usando un **diagramma Use Case (UCD)** di alto livello.

4.1 ATTORI

Gli attori sono rappresentati come una **figura stilizzata**.



Rappresentano un'entità esterna che interagisce con il **Sistema Sotto Sviluppo (SUD)**.

Ogni attore ha un nome unico.

Sono più grossolani delle Personas, infatti un Attore può essere associato a più Personas.

4.2 USE CASES

Sono rappresentati come ellissi denominati.



Corrispondono a features offerte dal sistema, provvedendo a benefit e utilità agli attori.

Modellano i requisiti funzionali.

Uno use case astrae più possibili scenari (sequenze di azioni) per una data feature.

Uno Use Case può essere visto come la classe di uno scenario.

4.3 EURISTICHE PER L'IDENTIFICAZIONE DEGLI ATTORI

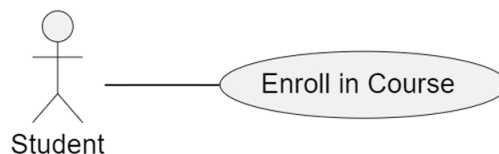
Ci dovremmo chiedere:

- Quale gruppo di utenti è supportato dal SUD nel loro lavoro?
- Quale gruppo di utenti esegue una funzione secondario del SUD, come l'amministrazione?
- Il SUD interagisce con sistemi o software esterni?

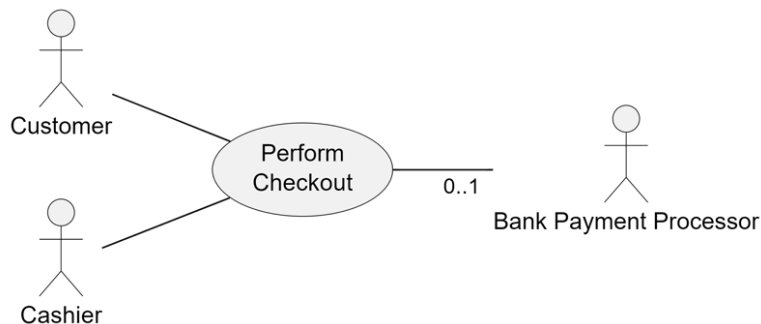
Gli attori non corrispondono una singola entità, ma rappresenta una classe di utenti che ha un determinato ruolo.

4.4 DIAGRAMMA USE CASE: ASSOCIAZIONI

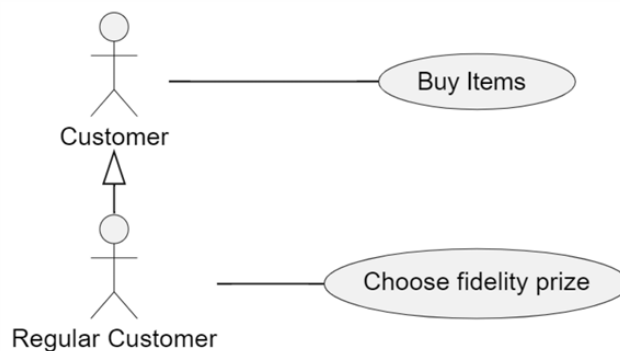
Un'associazione tra un attore e uno use case indica che l'attore può eseguire lo use case. Graficamente è rappresentata in questo modo:



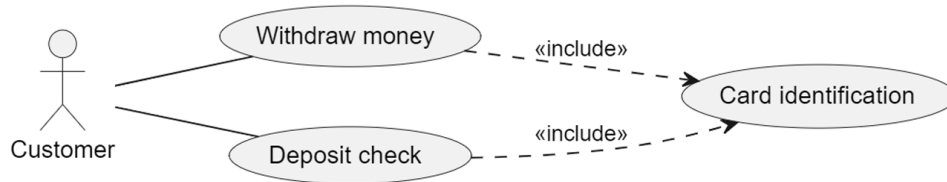
Uno Use Case può essere associato a più attori. Il significato è che più attori **collaborano** in qualche modo per eseguire quello use case. Graficamente:



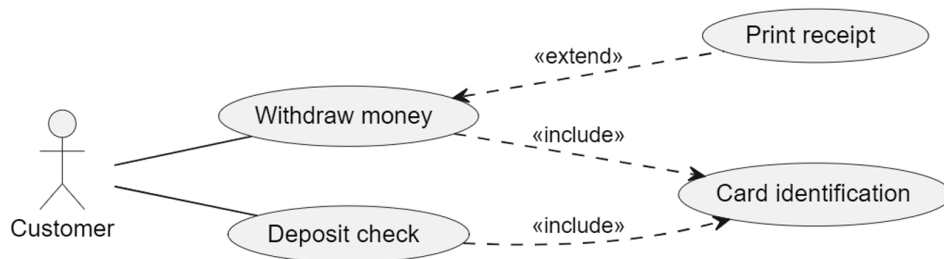
La **generalizzazione tra gli attori** può essere supportata quando uno degli attori è un sottotipo di un altro attore. Gli attori "specializzati" possono eseguire tutti gli use case che l'attore "padre" esegue. Graficamente:



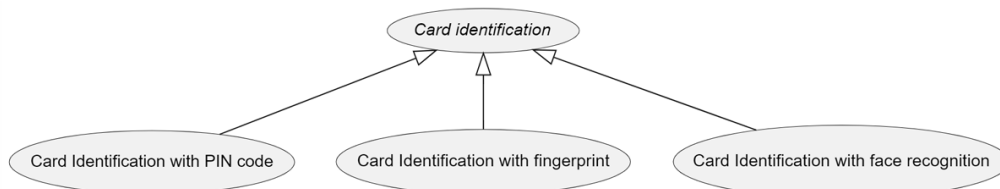
La **relazione** <<include>> è pensata per essere utilizzata quando due o più use case presentano parti di comportamento in comune. Le parti che includono dipendono dalle parti incluse per essere significative. Graficamente:



La **relazione** <<extend>> è pensata per essere utilizzata quando è presente un comportamento aggiuntivo che può essere aggiunto, eventualmente in modo condizionale, al comportamento definito in uno o più casi d'uso. Graficamente:



È possibile anche effettuare una **generalizzazione tra gli use case**. Graficamente:



4.5 FORMATI DI USE CASE

Gli Use Case possono essere scritti in diversi formati e livelli di formalità. Uno di questi è il **fully-dressed description**, in cui ogni step e variazione vengono scritte nel dettaglio.

Un template fondamentale è quello di **Cockburn**.

USE CASE #X	Name of the Use Case			
Goal in Context	Description of the objective of this UC			
Preconditions	All the conditions that must apply to start the UC			
Success End Condition	State of the system if the UC was successful			
Failed End Condition	State of the system if the UC failed			
Primary Actor	Primary actor of the UC			
Trigger	Action of the primary actor that initiates the UC			
Main Scenario	Step n.	Actor 1	Actor n	System
	1	Trigger action		
	2			Response
	..	Action 2		
	..		...	...
	n			Final action

Extension #1 (short description)	Step	Actor 1	Actor n	System
	x <condition>	...	...	...
	...	..	...	...
	...			Final action (possibly return to a step of the main scenario)
Extension #n (short description)	Step	Actor 1	Actor n	System
	y <condition>	...	...	...
	...	..	...	...
	...			Final action
Open Issues	List all the aspects that still need to be clarified. At the delivery of the doc must be empty			

4.5.1 Main Scenario ed Extensions

Lo scenario principale è la sequenza di azioni che si verifica quando tutto nello Use Case procede senza intoppi come previsto. Tuttavia, potrebbero esserci diversi modi per eseguirlo:

- Gli utenti possono autenticarsi utilizzando il PIN o uno scanner di impronte digitali;
- Potrebbe verificarsi un errore a un certo punto.

Nel definire il comportamento funzionale del sistema, è importante descrivere anche queste sequenze alternative di azioni che possono verificarsi durante l'esecuzione di uno Use Case. Questo viene fatto utilizzando le **extentions**.

CAPITOLO 5

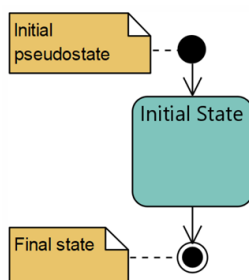
UML Statecharts

Un diagramma di stato descrive il comportamento di un sistema, di una parte di un sistema o di un oggetto specifico. Un UML Statecharts contiene:

- Una **regione** al livello superiore;
- Ogni regione contiene **vertici** (rappresentano uno stato) e **transizioni** (rappresentate come bordi diretti tra due vertici);

5.1 STATI

Uno **stato** è rappresentato da un rettangolo arrotondato con il suo nome. In un dato momento, può verificarsi un solo stato del diagramma. Tutti gli stati sono considerati stabili: solo un evento può causare la transizione da uno stato all'altro.

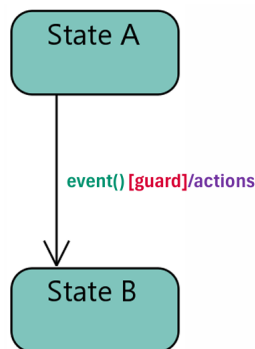


5.2 TRANSIZIONI

Le transizioni indicano cambiamenti di stato. Possono essere decorati con un label nella forma `triggers[guard]/actions` dove:

- `triggers` è la lista di eventi che possono portare al cambiamento di stato;
- `guard` è una condizione booleana;
- `actions` è una lista di operazioni da eseguire quando la transizione si attiva.

Queste parti sono opzionali e sono possibili auto-transizioni.



Affinché una transizione sia attivabile:

- Devono essere attivati tutti gli eventi corrispondenti ai trigger;
- La condizione in guard deve risultare VERA.

Una **transizione spontanea** è quella con nessun trigger e nessuna guard.

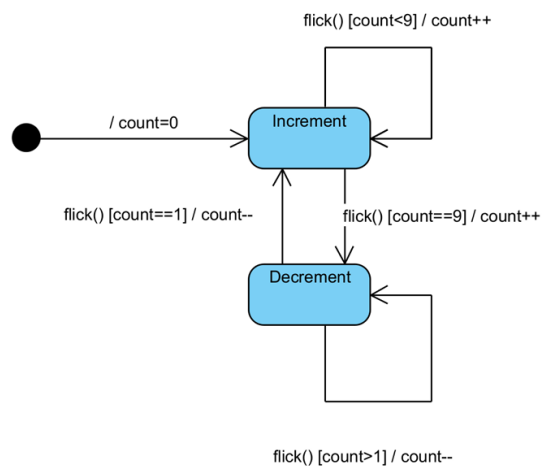
Vediamo un esempio:

```

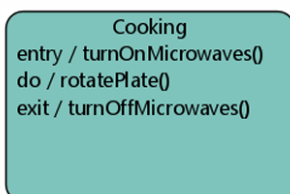
public class Counter {
    private int count = 0;
    private String mode = "increment";
    public void flick() {

        if(mode.equals("increment"))
            count++;
        else
            count--;

        if(count==10)
            mode = "decrement";
        else if (count==0)
            mode = "increment";
    }
}
  
```



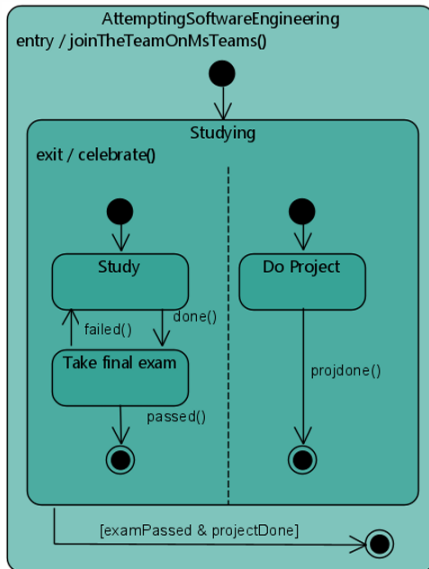
5.3 STATI: ATTIVITÀ INTERNE



Gli stati possono, opzionalmente, contenere una lista di **attività interne**. Ogni attività è caratterizzata da un label che indica quando l'attività deve essere invocata. In particolare:

- **entry/**: attività svolta all'ingresso;
- **do/**: attività svolta finché il sistema è nello stato;
- **exit/**: attività svolta all'uscita.

5.4 STATI COMPOSTI: REGIONI PARALLELE



Gli stati composti includono diagrammi Statechart all'interno dei singoli stati.

Gli stati composti possono contenere più regioni, che rappresentano comportamenti che possono verificarsi in parallelo.

Quando si esce da uno stato composto, tutte le sue regioni vengono terminate.

Aggiungi Shallow History Pseudostates.

Design del Software

Vediamo le regole per un buon design del software.

6.1 COESIONE

La **coesione** è una misura di quanto strettamente correlate e focalizzate siano le responsabilità (o i servizi offerti) di un modulo.

Se un'unità è responsabile di una sola task, allora possiamo dire che quella unità ha **alta coesione**.

Si può avere:

- **Coesione di metodo:** un metodo dovrebbe essere responsabile di un'unica task ben definita;
- **Coesione di classe:** ogni classe dovrebbe rappresentare un singolo concetto ben definito.

Il **Single Responsibility Principle** afferma che:

- Una classe dovrebbe avere una sola responsabilità;
- Una classe dovrebbe avere un solo motivo per essere modificata;
- Una responsabilità è un motivo per cambiare;
- Se una classe ha più di una responsabilità, può diventare vulnerabile quando uno qualsiasi dei requisiti cambia.

I benefici della coesione sono:

- **Riutilizzo:** se tutte le tue classi rispettano il principio SRP, è più probabile che tu possa riutilizzarne alcune;
- **Chiarezza:** il codice risulta più pulito, poiché le classi non compiono azioni impreviste;
- **Denominazione:** poiché tutte le tue classi hanno un'unica responsabilità, scegliere un nome appropriato è facile;
- **Leggibilità:** grazie alla maggiore chiarezza, ai nomi più appropriati e alle dimensioni ridotte dei file, la leggibilità è notevolmente migliorata.

6.2 ACCOPPIAMENTO

L'**accoppiamento** è una misura che indica quanto una classe sia collegata ad altre classi, ne conosca le caratteristiche o dipenda da esse.

Si dice che due o più classi siano accoppiate quando è impossibile riutilizzarne una senza riutilizzare anche l'altra o le altre.

Per la qualità del codice, dovremmo puntare ad avere basso accoppiamento.

Un'ottima soluzione è utilizzare le **interfacce**.

6.3 LA LEGGE DI DEMETRA

Si può riassumere con una regola d'oro: "Non parlare agli sconosciuti" (Don't talk to strangers).

Il principio stabilisce che un determinato oggetto dovrebbe assumere il meno possibile riguardo alla struttura o alle proprietà di qualsiasi altro oggetto. In termini pratici, un componente software dovrebbe interagire solo con i suoi "amici stretti" e non navigare attraverso l'intera struttura interna degli oggetti per ottenere ciò che gli serve.

Formalmente, la Legge di Demetra afferma che un metodo M di un oggetto O può invocare solo i metodi di:

- L'oggetto O stesso.
- Gli oggetti passati come parametri al metodo M.
- Qualsiasi oggetto creato o istanziato all'interno del metodo M.
- I componenti diretti di O (ovvero le sue variabili di istanza/attributi).

6.4 CRC CARDS

Una **card CRC** è una card associata a ciascuna classe, che rappresenta i seguenti concetti:

- Nome della classe;
- Responsabilità della classe (le conoscenze che la classe gestisce o i servizi che la classe fornisce);
- Collaboratori della classe (altre classi le cui competenze o i cui servizi devono essere sfruttati per adempiere alle responsabilità sopra indicate).

Class Name	
Superclasses	
Subclasses	
Responsibilities	Collaborators

Architetture Software Moderne

L'architettura non è semplicemente uno schema tecnico, ma un "blueprint" per la costruzione e l'evoluzione di un sistema. Riguarda le decisioni che hanno un impatto significativo e che sono difficili da cambiare in seguito (es. scelta tra database relazionale o NoSQL).

7.1 CARATTERISTICHE INFLUENZATE DALL'ARCHITETTURA

7.1.1 Scalabilità

Un servizio è **scalabile** se l'aumento delle prestazioni è proporzionale alle risorse aggiunte. Un problema di performance rende il sistema lento per un singolo utente; un problema di scalabilità rende il sistema lento solo sotto carico pesante.

Tipologia di scaling sono:

- **Orizzontale**: Aumento del numero di nodi di calcolo.
- **Verticale**: Aumento della potenza di calcolo di un singolo nodo.
- **Diagonale**: Combinazione dei due approcci precedenti.

7.1.2 Bilanciamento del carico (Load Balancing)

Nelle architetture scalabili orizzontalmente, il **Load Balancer** distribuisce il traffico. Le strategie includono:

- **Random** (il traffico in entrata viene indirizzato in modo casuale verso uno dei nodi);
- **Meno trafficato** (traffico indirizzato al nodo meno trafficato);
- **Round robin**;
- **Livello 4** (controllare i dettagli del livello di trasporto (ad es.: indirizzo IP di origine, ...));
- **Layer 7** (look at application layer details (e.g.: cookies, sessions, ...));

Nota critica: Il load balancer può diventare un collo di bottiglia o un singolo punto di fallimento (SPOF).

7.1.3 Disponibilità (Availability)

La **disponibilità** è la percentuale di tempo in cui un sistema o un servizio è operativo e accessibile. Misurata come percentuale di uptime (il "9"), definisce quanto tempo un sistema è operativo. Ad esempio, cinque 9 (99,999%) equivalgono a circa 5 minuti di downtime all'anno.

Come si ottiene alta disponibilità?

- **Fail-over attivo-passivo**: un'istanza secondaria subentra solo in caso di guasto della primaria;
- **Fail-over attivo-attivo**: più istanze lavorano simultaneamente; se una fallisce, le altre assorbono il carico.

7.2 GESTIONE DEI DATI: RDBMS VS NOSQL

Fare lo scaling orizzontale di un'istanza di un'applicazione era relativamente semplice, in quanto bastava prendere un nuovo server con l'applicazione installata e bilanciare il carico. Molte applicazioni però richiedono anche la persistenza dei dati. E se dovessimo scalare anche i **Database Relazionali**? Non è così semplice.

L'**ACID** è un insieme di proprietà delle transazioni nei database relazionali.

- **Atomicità**: ogni transazione è di tipo "tutto o niente";
- **Coerenza**: ogni transazione porta il database da uno stato valido all'altro;
- **Isolamento**: l'esecuzione simultanea delle transazioni produce gli stessi risultati che si otterrebbero se le transazioni fossero eseguite in modo seriale;
- **Persistenza (Durability)**: una volta che una transazione è stata confermata, rimarrà tale.

Le proprietà ACID rendono impossibile scalare un RDBMS semplicemente in orizzontale. Lo si fa o **verticalmente** (aggiungendo più risorse di calcolo), oppure tramite specifiche tecniche:

- **Master-Slave**: il Master gestisce letture/scritture, gli Slave sono repliche in sola lettura;
- **Master-Master**: tutte le istanze servono letture e scritture (alto overhead di sincronizzazione);
- **Federazione**: divisione del database per funzione (es. database di utenti, database di prodotti);
- **Sharding**: distribuisce i dati su più database in modo tale che ciascun database gestisca solo un sottoinsieme dei dati;
- **Denormalizzazione**: Ridondanza dei dati per migliorare le performance di lettura (es. viste materializzate).

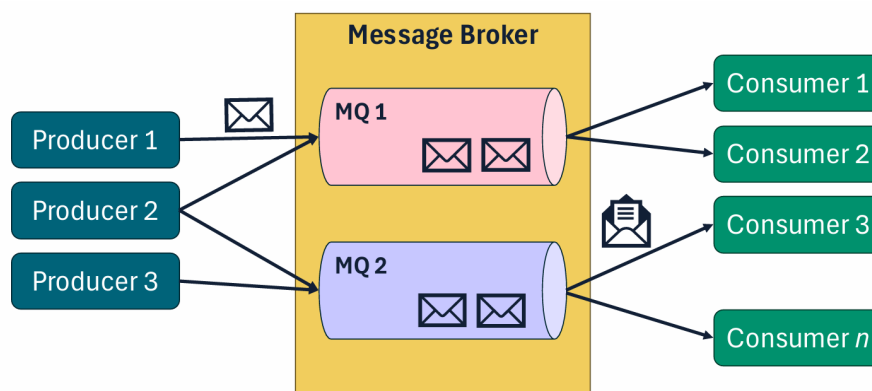
I database **NoSQL** rinunciano alla consistenza forte a favore della scalabilità, adottando il modello **BASE**:

- **Basically Available**: il sistema garantisce la disponibilità;
- **Soft state**: lo stato del sistema può variare nel tempo, anche in assenza di input;
- **Eventual consistency**: il sistema raggiungerà la coerenza nel corso di un determinato periodo di tempo, a condizione che non riceva input durante tale periodo.

7.3 ASINCRONISMO E SISTEMI DI MESSAGGISTICA

Per ridurre la latenza percepita dall'utente, le operazioni lunghe vengono gestite asincronamente tramite **Code di Messaggi** (Message Queue) (MQ):

- Un'applicazione pubblica un messaggio su una MQ, con i dettagli dell'attività da eseguire;
- Un **worker** (in modo asincrono, possibilmente in un secondo momento) agisce come consumatore, preleva il messaggio dalla coda MQ ed esegue effettivamente l'attività;
- L'utente non rimane bloccato in attesa del completamento dell'attività lunga;
- L'attività viene effettivamente completata in background, quando uno dei possibili worker diventa disponibile.



Un **Message Broker** (o intermediario di messaggi) è un componente architetturale del software che facilita la comunicazione e lo scambio di dati tra diverse applicazioni, sistemi o servizi.

In questo genere di code un messaggio esiste solo per essere elaborato una volta. Quando un consumatore legge un messaggio e conferma di averlo processato, il broker lo cancella definitivamente dalla coda.

Alcuni Message Brokers supportano gli **streams**. Funzionano come un libro mastro o un registro storico. I nuovi messaggi vengono continuamente aggiunti alla fine (append-only). Quando un consumatore legge un messaggio, il messaggio non viene cancellato. Rimane nello stream fino a quando non scade un certo limite di tempo impostato (es. 7 giorni) o non si riempie lo spazio su disco.

7.4 MICROSERVIZI

L'architettura a microservizi struttura il software come un insieme di servizi indipendenti organizzati attorno a capacità di business.

- Nell'architettura **monolita** ci sono diversi componenti/moduli integrati in un'unica unità. Facilità di dispiegamento iniziale, ma diventano difficili da mantenere e scalare all'aumentare della complessità.
- Nell'architettura a **microservizi** il software è strutturato come un insieme di servizi.
 - Ogni servizio ha il proprio codice sorgente;
 - Possibilmente poliglotta;
 - Ogni servizio può essere implementato in modo indipendente;

Dividere un monolite in un certo numero di microservizi, aiuta a rendere più gestibili grandi sistemi. Questo perché ogni microservizio è significativamente più piccolo di un intero sistema, oltre ad essere più facili da comprendere, da mantenere, ecc...

I monoliti raggruppano tutte le funzionalità in un unico processo e possono essere **scalati replicando** il monolite su più server. I microservizi, invece, collocano ogni funzionalità in processi diversi, e garantiscono la **scalabilità distribuendo** servizi su server differenti e replicandoli quando necessario.

7.4.1 Design

Scegliere un sistema a microservizi ha senso solo se si ha a che fare con sistemi complessi.

Il design è basato su **Domain-Driven Design (DDD)**. In linea generale, bisogna ricordarsi che un servizio dovrebbe fare *una sola cosa*. In particolare, il design è strutturato nei seguenti passaggi:

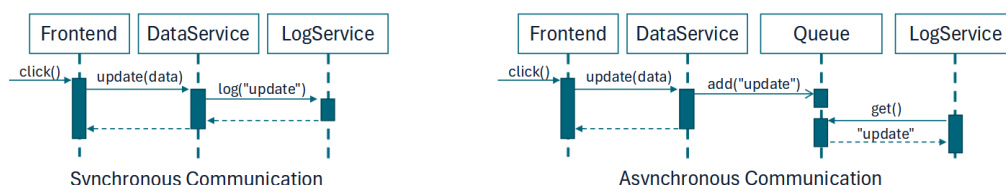


1. **Analisi del Dominio**: comprendere i requisiti funzionali;
2. **Definizione dei Bounded Context**: ogni contesto contiene un modello di dominio specifico;
3. **Entità, Aggregati e Servizi**: definire oggetti con identità e confini di consistenza transazionale;
4. **Identificazione dei Microservizi**: un microservizio non dovrebbe mai estendersi oltre un singolo Bounded Context.

7.4.2 Comunicazione

La comunicazione può essere:

- **Sincrona**: con REST o Apache Thrift. Le chiamate sincrone eccessive possono causare un effetto moltiplicativo dei tempi di inattività (downtime);
- **Asincrona**: con le code di messaggi (MS).



7.4.3 Gestione dei dati

I monoliti sono spesso associati a un **singolo, centralizzato database**.

I microservizi possono avere un database condiviso. Si raccomanda però **un database per servizio** per garantire l'indipendenza dei team e consentire la "persistenza poliglotta" (ogni servizio usa il DB più adatto).

7.5 ORGANIZZAZIONE DEL TEAM DI ARCHITETTURA E SVILUPPO

L'architettura del software riflette inevitabilmente la struttura di comunicazione dell'organizzazione che lo produce. Infatti, la **legge di Conway** afferma che:

"Le organizzazioni che progettano sistemi sono costrette a produrre progetti che sono copie delle strutture di comunicazione di queste organizzazioni."

L'esempio classico che viene spesso citato per spiegare questo concetto è: se assigni la costruzione di un compilatore a quattro team diversi, otterrai inevitabilmente un compilatore a quattro passaggi.

Oggi, nell'ingegneria del software moderna, la Legge di Conway non viene vista solo come un vincolo da subire, ma come uno strumento strategico.

Le aziende più all'avanguardia applicano la cosiddetta **Manovra di Conway Inversa** (Inverse Conway Maneuver). Invece di sperare che l'organizzazione attuale riesca magicamente a produrre l'architettura software desiderata, fanno l'esatto opposto: ristrutturano l'azienda e i team di sviluppo in modo che ricalchino l'architettura software che vogliono ottenere. Se il tuo obiettivo è passare da un vecchio software monolitico a un'architettura moderna a microservizi scalabili, il primo passo non è scrivere codice nuovo, ma smembrare i vecchi reparti e creare team piccoli e indipendenti. Il software seguirà a ruota.

CAPITOLO 8

La virtualizzazione nell'implementazione di sistemi complessi

La maggior parte dei programmi è scritta per essere eseguita in modo sequenziale su una singola CPU. Con l'avvento dei processori multi-core, la sfida principale è diventata come utilizzare in parallelo le molteplici unità di calcolo disponibili. La **virtualizzazione** risponde a questa sfida simulando l'esistenza di più computer indipendenti sulla stessa macchina fisica.

8.1 ARCHITETTURA DELLA MACCHINA VIRTUALE (VM)

Una macchina virtuale è una risorsa creata virtualmente che simula un server fisico completo. L'elemento centrale è l'**Hypervisor**, un nuovo strato software che viene eseguito prima del sistema operativo.

Le caratteristiche chiave di una VM sono:

- **Indipendenza dall'Hardware:** l'Hypervisor simula una configurazione standard, rendendo le VM portabili;
- **VM come File:** una macchina virtuale è essenzialmente un file che contiene il sistema operativo, le applicazioni, i dati e lo stato della memoria. Questo permette di copiare o clonare interi server con estrema facilità;
- **Isolamento Totale:** ogni VM è isolata dalle altre sullo stesso server. Un attacco informatico o un crash di sistema su una VM non influisce sulle altre;
- **Gestione Centralizzata:** l'intero data center può essere amministrato da un'unica console, semplificando drasticamente la gestione.

8.2 BENEFICI DEL CONSOLIDAMENTO

Il passaggio alla virtualizzazione porta a quello che viene definito "**consolidamento**" del data center.

- **Ottimizzazione delle Risorse:** esecuzione di sistemi operativi diversi simultaneamente sulla stessa macchina;
- **Efficienza dello Spazio Disco:** alle VM viene allocato solo lo spazio strettamente necessario; il resto rimane disponibile per nuovi carichi;
- **Semplificazione del Disaster Recovery:** essendo file, il backup e il ripristino dell'intero stato del server (inclusi i dispositivi e lo stato operativo) sono estremamente semplificati.
- **Riduzione dei Costi:** risparmi significativi su hardware, energia elettrica, condizionamento, spazio fisico e costi del personale IT grazie alla gestione centralizzata.

8.3 CLOUD COMPUTING

Il **Cloud Computing** rappresenta l'evoluzione finale della virtualizzazione, fornendo risorse IT (calcolo, storage, database) come servizio attraverso data center geograficamente distribuiti.

Le caratteristiche fondamentali sono:

- **Scalabilità:** caratteristica di un sistema, di una rete o di un processo, che indica la sua capacità di gestire con efficacia carichi di lavoro crescenti o di essere facilmente scalabile;
- **Elasticità:** la capacità di aumentare o ridurre le risorse infrastrutturali in modo dinamico, in base alle necessità, per adattarsi in modo autonomo alle variazioni del carico di lavoro, ottimizzando l'utilizzo delle risorse.

Come possiamo ottenere queste proprietà? Attraverso l'**Allocazione dinamica delle risorse** (detta anche Provisioning Dinamico) e l'**Architettura Multi-tenant**.

Il Provisioning Dinamico consiste nella capacità del sistema cloud di assegnare (allocare) e revocare (deallocare) risorse computazionali (come CPU, RAM, spazio di archiviazione o interi server virtuali) in modo completamente automatico e in tempo reale, in base alle necessità del momento.

L'Architettura Multi-tenant consiste nel progettare un'infrastruttura o un software in cui una singola istanza dell'applicazione gira su un server, ma serve più clienti diversi (chiamati tenant, ovvero "locatari" o "inquilini").

8.4 MODELLI DI SERVIZI

I modelli di servizio Cloud definiscono il livello di risorse, astrazione e controllo che viene fornito all'utente finale. Si dividono principalmente in tre categorie: **IaaS**, **PaaS** e **SaaS**.

8.4.1 Infrastructure as a Service - IaaS

Nel modello IaaS, il provider ti affitta l'hardware "nudo e crudo", ma virtualizzato. Ti fornisce server, spazio di archiviazione (storage) e reti. Tutto il resto spetta a te.

- **Cosa gestisce il provider:** server fisici, rete, dischi rigidi, virtualizzazione;
- **Cosa gestisci tu:** il sistema operativo (es. decidi tu se installare Linux o Windows), i database, le applicazioni, la sicurezza dei dati;
- **Esempi:** Google Compute Engine, Amazon EC2 (Elastic Compute Cloud), Microsoft Azure VMs.

8.4.2 Platform as a Service - PaaS

Saliamo di un gradino. Nel PaaS, il provider ti dà non solo l'infrastruttura, ma anche l'ambiente già pronto per farti lavorare. Non devi preoccuparti di installare o aggiornare il sistema operativo o di configurare il server web: è tutto pre-configurato.

- **Cosa gestisce il provider:** hardware, virtualizzazione, sistemi operativi, middleware, runtime e ambienti di sviluppo;
- **Cosa gestisci tu:** solo ed esclusivamente il codice della tua applicazione e i tuoi dati;
- **Esempi:** Google App Engine, Heroku, AWS Elastic Beanstalk.

8.4.3 Software as a Service - SaaS

Questo è il livello più alto e quello che usiamo tutti noi ogni giorno. Nel SaaS, l'intero prodotto è gestito dal provider ed è pronto all'uso, di solito accessibile tramite un browser web.

- **Cosa gestisce il provider:** assolutamente tutto. Hardware, sistema operativo, applicazione, database, aggiornamenti e sicurezza;
- **Cosa gestisci tu:** Niente, se non il tuo account utente (es. la tua password o le tue preferenze nell'app);
- **Esempi:** Gmail, Netflix, Spotify, Microsoft 365, Google Workspace (Docs, Drive, ecc.).

8.5 MODELLI DI IMPLEMENTAZIONE

Esistono tre modelli principali di implementazione nel cloud: **Public Cloud**, **Private Cloud** e **Hybrid Cloud**.

8.5.1 Public Cloud

In questo modello, l'infrastruttura cloud è resa disponibile al pubblico generale o a un vasto gruppo industriale. È di proprietà di un'organizzazione (il provider) che vende i servizi cloud. Conosciuto anche come cloud esterno o multi-tenant, è un ambiente apertamente accessibile a tutti tramite internet.

8.5.2 Private Cloud

L'infrastruttura cloud è operata esclusivamente per una singola organizzazione. Può essere gestita direttamente dall'azienda stessa o da una terza parte, e i server possono trovarsi fisicamente all'interno dell'azienda (on-premise) oppure all'esterno.

8.5.3 Hybrid Cloud

Questo modello è una composizione di due o più cloud (pubblici o privati) che rimangono entità uniche e separate, ma che sono interconnesse tramite tecnologie standardizzate o proprietarie.

CAPITOLO 9

Il Paradigma REST

Le moderne applicazioni sono frequentemente suddivise in due componenti principali che comunicano via Internet:

- **Backend**: responsabile dei dati e della logica business (lato server);
- **Frontend**: responsabile della UI (lato client).

Le **Application Programming Interfaces (APIs)** rappresentano il mezzo attraverso cui questi programmi comunicano. Storicamente, sono stati utilizzati protocolli dedicati come CORBA, Java RMI o SOAP, che spesso risultavano costosi da implementare e poco interoperabili. L'approccio moderno consiste nell'utilizzare direttamente i protocolli web standard, in particolare l'**HTTP**.

9.1 FONDAMENTI DELLO STILE ARCHITETTONICO REST

REST non è uno standard né un protocollo, è uno **stile architetturale**, un insieme di principi e linee guida che definiscono come gli standard dovrebbero essere usati nelle API. È basato su HTTP e URIs.

Facciamo uno sguardo all'**HTTP**. Sta per HyperText Transfer Protocol e ha due tipi di messaggi: Richiesta e Risposta.

Le richieste HTTP si basano sull'idea del CRUD (Create, Retrieve, Update, Delete) e le più comuni sono:

- **POST**: salva e crea una nuova risorsa;
- **GET**: recupera informazioni riguardo una risorsa;
- **PUT**: aggiorna le informazioni riguardo una risorsa;
- **DELETE**: elimina la risorsa;

Le risposte HTTP hanno degli **status code** che danno informazioni riguardo lo stato della richiesta effettuata. I più comuni sono:

- **200**: richiesta è stata correttamente gestita;
- **201**: una risorsa è stata correttamente creata;
- **400**: non è possibile comprendere la richiesta;
- **401**: autenticazioni fallita, o non autorizzata;
- **404**: risorsa non trovata;

- 405: metodo non supportato dalla risorsa;
- 500: errore di applicazione.

Torniamo ora a REST. Si basa su i seguenti concetti chiave:

- **Risorsa:** qualsiasi entità sufficientemente importante da essere referenziata come "cosa in sé". Solitamente corrisponde a oggetti di dominio persistenti;
- **URI (Uniform Resource Identifier):** ogni risorsa deve essere associata a un URI univoco;
- **Statelessness:** l'API non deve utilizzare sessioni lato server. Ogni richiesta deve contenere tutte le informazioni necessarie per essere elaborata, il che migliora drasticamente la scalabilità;
- **Rappresentazione:** lo stato della risorsa viene trasferito tra client e server tramite formati di interscambio dati, principalmente JSON o XML.

Esempi:

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todos/<ID>	Retrieve only the To-do item whose ID is <ID>
POST /todos	Save a new To-do item. Data of the exam to save are in the request body
PUT /todos/<ID>	Replace (or create) the To-do item whose ID is <ID>, using the data in the request body
DELETE /todos/<ID>	Delete the To-do item having ID <ID>

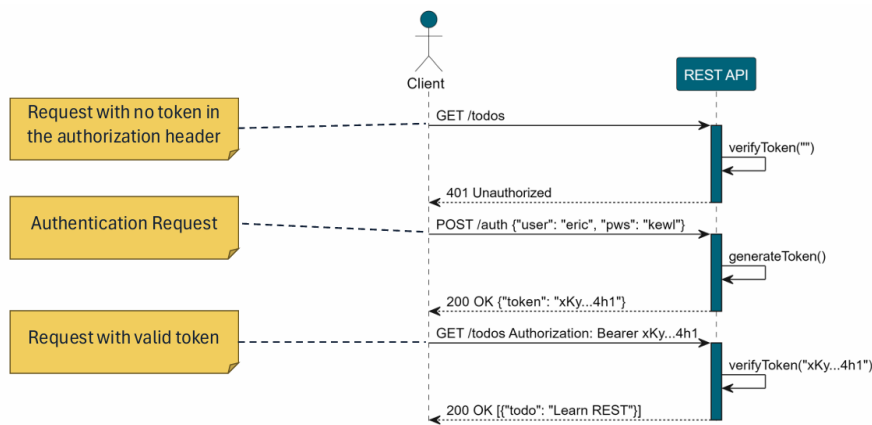
Nelle API reali, le risorse possono presentare gerarchie (es. un'azienda ha dipartimenti che hanno dipendenti). REST permette di definire risorse annidate (es. GET /listings/id/reviews).

Per quanto riguarda il filtraggio e l'ordinamento dei dati, è fondamentale rimanere **REST compliant**:

- **Errore comune:** Creare nuovi percorsi come /todosSortedAsc o /todos/filter/key. Questo non è conforme a REST e complica l'API;
- **Pratica corretta:** Utilizzare *query parameters* su un unico percorso base.
 - Esempio ordinamento: GET /todos?sort=title&mode=asc
 - Esempio ricerca: GET /todos?q=key

9.2 SICUREZZA DELLE API: AUTENTICAZIONE E JWT

La sicurezza si divide in due ambiti: **Autenticazione** (identità dell'utente) e **Autorizzazione** (permessi di accesso alle risorse). Lo schema più diffuso è quello basato sui **Token**.



Il **JSON Web Token** (JWT) è uno standard aperto per la condivisione sicura di "claims" tra due parti. È composto da tre parti separate da un punto (.):

- **Header:** contiene il tipo di token e l'algoritmo di hashing (es. HS256). Codificato in Base64Url;
- **Payload:** contiene i dati (claims), come il nome utente o la data di scadenza (exp). Codificato in Base64Url;
- **Signature:** creata applicando una funzione crittografica di hashing all'header e al payload concatenati, utilizzando una chiave segreta nota solo al server.

Una funzione di crittografia di hashing è una funzione che mappa una stringa binaria arbitraria (input) su una stringa binaria di lunghezza fissa (digest o hash). Sono *collision free*, ovvero è impossibile trovare due input diversi che portino allo stesso hash. Si possono calcolare facilmente, ma non è possibile fare l'operazione inversa (quindi andare da hash a input).

9.3 IMPLEMENTARE UNA REST API

L'implementazione di un'API REST da zero richiederebbe sforzi considerevoli per il parsing delle richieste, la serializzazione/deserializzazione di JSON/XML e la gestione del routing. Per questo si utilizzano i **Web Framework**.

I framework sono diversi dalle librerie in quanto:

- **Libreria:** un insieme di funzioni chiamate dal codice utente per eseguire compiti specifici. Il controllo del flusso rimane all'utente.
- **Framework:** Una collezione di componenti riutilizzabili che incarna un design astratto. Si basa sull'**Inversion of Control** (IoC), cioè è il framework che gestisce il flusso di esecuzione e chiama il codice dell'utente quando appropriato.

Un framework può essere:

- **Opinionato (es. Spring, Django, Ruby on Rails):** impongono convenzioni rigide e decisioni predefinite;
 - *Pro*: alta coerenza tra progetti, velocità di sviluppo, riduzione della fatica decisionale;
 - *Contro*: curva di apprendimento ripida, minore flessibilità.
- **Non Opinati (es. Flask, Express, Koa):** offrono maggiore libertà e flessibilità nelle scelte architettoniche.

CAPITOLO 10

Design Pattern

Un **Design Pattern** è una soluzione comune a un problema ricorrente nel design del software. Esso astrae una struttura di progettazione specifica, comprendendo classi, oggetti, dipendenze e interazioni.

Ogni pattern è definito da quattro elementi:

- **Nome:** identifica il pattern in modo univoco;
- **Problema:** descrive il contesto in cui applicare il pattern;
- **Soluzione:** definisce la struttura (micro-architettura) e i termini concreti;
- **Conseguenze:** analizza i compromessi e i risultati dell'applicazione.

10.1 CLASSIFICAZIONE DEI PATTERN GOF (GANG OF FOUR)

I pattern classici sono suddivisi in tre categorie principali in base al loro scopo:

Categoria	Focus Principale	Obiettivo
Creazionali	Creazione di oggetti complessi	Nascondere i dettagli di istanziazione e assemblaggio.
Strutturali	Composizione di classi e oggetti	Creare strutture più ampie partendo da componenti indipendenti
Comportamentali	Algoritmi e responsabilità	Gestire le interazioni e la distribuzione dei compiti tra oggetti

10.2 PATTERN CREAZIONALI

10.2.1 Singleton

Garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale. Utilizza un costruttore privato e un metodo statico per il recupero dell'istanza (es. per connessioni a database).

È il tipico approccio che si usa (o si usava) per gestire risorse condivise, come ad esempio la connessione a un database, un logger di sistema o la configurazione globale dell'applicazione. Non ha senso, e anzi sarebbe uno spreco di memoria, creare 100 istanze diverse del logger; ne basta una che tutti possono chiamare.

L'implementazione classica si basa su tre regole fondamentali che "blindano" la classe. Ecco un esempio concettuale (scritto in Java):

```
1 public class Singleton {
2     // 1. Una variabile statica privata per conservare l'unica istanza
3     private static Singleton instance;
4
5     // 2. Un costruttore privato per impedire l'uso di 'new' dall'esterno
6     private Singleton() {
7         // Inizializzazione
8     }
9
10    // 3. Un metodo statico pubblico che agisce da costruttore globale
11    public static Singleton getInstance() {
12        if (instance == null) {
13            instance = new Singleton(); // Creata solo la prima volta (Lazy
14            Initialization)
15        }
16        return instance;
17    }
18 }
```

Oggi, l'approccio corretto e preferito per avere "una sola istanza" non è usare il pattern Singleton, ma delegare la creazione dell'oggetto a un container di Dependency Injection (come Spring in Java o i container di .NET).

10.2.2 Builder

L'intento principale del Builder è separare la costruzione di un oggetto complesso dalla sua rappresentazione, permettendo allo stesso processo di costruzione di creare rappresentazioni diverse.

Immagina di dover creare un oggetto *Utente*. Un utente ha nome e cognome (obbligatori), ma poi ha una serie di attributi opzionali: età, indirizzo, numero di telefono, bio, url del profilo.

Senza un Builder, ti scontreresti con l'anti-pattern del *Telescoping Constructor* (costruttore telescopico):

```
1 // L'incubo dei costruttori infiniti
2 public Utente(String nome, String cognome) { ... }
3 public Utente(String nome, String cognome, int eta) { ... }
4 public Utente(String nome, String cognome, int eta, String indirizzo) { ... }
5 // E così via...
```

Oppure saresti costretto a usare un costruttore gigantesco passando un sacco di null: `Utente u = new Utente("Mario", "Rossi", null, null, "12345678", null)`.

Il Builder risolve questo problema creando un oggetto intermedio (il costruttore, appunto) che raccoglie i parametri passo dopo passo e, alla fine, genera l'oggetto immutabile.

Un esempio:

```
1 public class Utente {
2     private final String nome;        // Obbligatorio
3     private final String cognome;    // Obbligatorio
4     private final int eta;           // Opzionale
5     private final String telefono;   // Opzionale
6
7     // Costruttore privato! L'unico modo per creare l'Utente e' usare il Builder
8     private Utente(UtenteBuilder builder) {
9         this.nome = builder.nome;
10        this.cognome = builder.cognome;
11        this.eta = builder.eta;
12        this.telefono = builder.telefono;
13    }
14
15    // --- CLASSE BUILDER ANNIDATA ---
16    public static class UtenteBuilder {
17        private final String nome;
18        private final String cognome;
19        private int eta = 0;           // Valore di default
20        private String telefono = null; // Valore di default
21
22        // I parametri obbligatori vanno nel costruttore del Builder
23        public UtenteBuilder(String nome, String cognome) {
24            this.nome = nome;
25            this.cognome = cognome;
26        }
27
28        // I parametri opzionali restituiscono 'this' (Fluent Interface)
29        public UtenteBuilder eta(int eta) {
30            this.eta = eta;
31            return this;
32        }
33
34        public UtenteBuilder telefono(String telefono) {
35            this.telefono = telefono;
36            return this;
37        }
38
39        // Il metodo finale che assembla e valida l'oggetto
40        public Utente build() {
41            // Qui puoi inserire logica di validazione complessa
42            // es: se metti l'eta', devi essere maggiorenne
43            return new Utente(this);
44        }
45    }
46 }
```

E l'utilizzo diventa elegante, leggibile e sicuro:

```
1 Utente mario = new Utente.UtenteBuilder("Mario", "Rossi")
2             .eta(30)
3             .telefono("123456789")
4             .build();
```

10.3 PATTERN STRUTTURALI

10.3.1 Adapter (o Wrapper)

Converte l'interfaccia di una classe in un'altra interfaccia attesa dal client, permettendo la collaborazione tra classi incompatibili.

Immagina che il tuo sistema di e-commerce sia progettato per usare questa interfaccia per i pagamenti in euro:

```
1 // L'interfaccia che il TUO sistema si aspetta (la presa europea)
2 public interface IProcessorePagamenti {
3     void paga(double euro);
4 }
```

Ora decidi di integrare Stripe, ma la libreria ufficiale di Stripe accetta solo pagamenti in dollari e ha un nome del metodo diverso:

```
1 // La libreria esterna (la spina americana, non puoi modificarla!)
2 public class StripeAPI {
3     public void makePayment(double dollars) {
4         System.out.println("Pagamento elaborato: $" + dollars);
5     }
6 }
```

Invece di riempire tutto il tuo codice di e-commerce con if/else e conversioni di valuta, crei un Adapter:

```
1 // L'Adattatore
2 public class StripeAdapter implements IProcessorePagamenti {
3
4     private StripeAPI stripe; // Composizione: l'Adapter "contiene" l'oggetto da
    adattare
5
6     public StripeAdapter(StripeAPI stripe) {
7         this.stripe = stripe;
8     }
9
10    @Override
11    public void paga(double euro) {
12        // L'Adapter fa il "lavoro sporco" di traduzione
13        double dollars = euro * 1.08; // Esempio di tasso di cambio
14        stripe.makePayment(dollars);
15    }
16 }
```

10.3.2 Composite

Compone gli oggetti in strutture ad albero per rappresentare gerarchie parte-tutto (es. file system con directory e file). Permette di trattare oggetti singoli e composizioni in modo uniforme.

Immagina di dover modellare il sistema di spedizione di un e-commerce. Un ordine può contenere dei singoli Prodotti (es. un telefono, un libro) e delle Scatole. Il trucco è che una Scatola può contenere Prodotti, ma può anche contenere altre Scatole più piccole, che a loro volta contengono prodotti, e così via.

Se il tuo obiettivo è calcolare il prezzo totale o il peso totale della spedizione, senza il Composite saresti costretto a scrivere un codice orribile pieno di controlli sui tipi:

```
1 // Esempio di come NON fare
2 if (oggetto instanceof Prodotto) {
3     totale += ((Prodotto) oggetto).getPrezzo();
4 } else if (oggetto instanceof Scatola) {
5     for (Object contenuto : ((Scatola) oggetto).getContenuto()) {
6         // E qui devi rifare i controlli! Un incubo ricorsivo.
7     }
8 }
```

Il Composite risolve la questione dichiarando un'interfaccia comune sia per gli oggetti semplici (le **Foglie**) sia per i contenitori (i **Nodi/Composite**).

Ecco come si struttura in Java:

```
1 // 1. L'Interfaccia Comune
2 public interface ComponenteSpedizione {
3     double calcolaPrezzo();
4 }
5
6 // 2. La "Foglia" (L'oggetto singolo che fa il lavoro vero)
7 public class Prodotto implements ComponenteSpedizione {
8     private double prezzo;
9
10    public Prodotto(double prezzo) { this.prezzo = prezzo; }
11
12    @Override
13    public double calcolaPrezzo() {
14        return prezzo;
15    }
16 }
17
18 // 3. Il "Composite" (Il contenitore che delega il lavoro ai figli)
19 public class Scatola implements ComponenteSpedizione {
20     // La scatola non sa se contiene Prodotti o altre Scatole.
21     // Sa solo che contiene "ComponentiSpedizione".
22     private List<ComponenteSpedizione> figli = new ArrayList<>();
23
24     public void aggiungi(ComponenteSpedizione componente) {
25         figli.add(componente);
26     }
27
28     @Override
29     public double calcolaPrezzo() {
30         double totale = 0;
31         // Il Composite si limita a iterare e delegare la chiamata. Magia della
32         // ricorsione!
33         for (ComponenteSpedizione figlio : figli) {
34             totale += figlio.calcolaPrezzo();
35         }
36         return totale;
37     }
38 }
```

Ora, il client (il codice che usa queste classi) non deve preoccuparsi di cosa sta manipolando. Chiama `calcolaPrezzo()` sulla scatola principale e l'albero si risolve da solo, a cascata.

10.3.3 Decorator

Aggiunge funzionalità a un oggetto in modo dinamico, offrendo un'alternativa flessibile all'ereditarietà (es. aggiungere ingredienti a una pizza base).

Immagina di dover sviluppare il software per la cassa di una caffetteria. Parti con una classe base Caffè. Poi il cliente ti chiede di aggiungere il latte, quindi crei la sottoclasse CaffèMacchiato. Poi vuole il caramello: crei CaffèAlCaramello. Fin qui tutto bene. Ma cosa succede se un cliente vuole un caffè macchiato e con il caramello? E se vuole anche l'aggiunta di panna?

Se usi l'ereditarietà classica, ti ritrovi a dover creare una combinazione infinita di classi: CaffèMacchiatoCaramello, CaffèPannaCaramello, CaffèMacchiatoPannaCaramello. È un incubo da mantenere (noto come *Class Explosion*).

Il Decorator risolve il problema "avvolgendo" l'oggetto originale dentro un altro oggetto (il decorator, appunto) che implementa la stessa identica interfaccia. Poiché hanno la stessa interfaccia, il client che li usa non si accorge della differenza.

Il decorator fa il suo lavoro (es. aggiunge il costo del caramello) e poi delega il resto del lavoro all'oggetto che ha avvolto.

```
1 // 1. L'Interfaccia base
2 public interface Bevanda {
3     double calcolaCosto();
4     String getDescrizione();
5 }
6
7 // 2. Il Componente Concreto (il nostro punto di partenza)
8 public class CaffèSemplice implements Bevanda {
9     @Override
10    public double calcolaCosto() { return 1.00; }
11
12    @Override
13    public String getDescrizione() { return "Caffè espresso"; }
14 }
15
16 // 3. I Decoratori (che implementano Bevanda MA contengono anche una Bevanda)
17 public class AggiuntaLatte implements Bevanda {
18     private Bevanda bevandaAvvolta;
19
20    public AggiuntaLatte(Bevanda bevanda) {
21        this.bevandaAvvolta = bevanda;
22    }
23
24    @Override
25    public double calcolaCosto() {
26        return bevandaAvvolta.calcolaCosto() + 0.50; // Aggiunge il suo costo
27    }
28
29    @Override
30    public String getDescrizione() {
31        return bevandaAvvolta.getDescrizione() + ", con latte";
32    }
33 }
34
35 public class AggiuntaCaramello implements Bevanda {
36     private Bevanda bevandaAvvolta;
37
38    public AggiuntaCaramello(Bevanda bevanda) {
39        this.bevandaAvvolta = bevanda;
40    }
41 }
```

```

42     @Override
43     public double calcolaCosto() {
44         return bevandaAvvolta.calcolaCosto() + 0.80;
45     }
46
47     @Override
48     public String getDescrizione() {
49         return bevandaAvvolta.getDescrizione() + ", al caramello";
50     }
51 }

```

E l'utilizzo diventa una vera e propria matrioska dinamica, componibile a runtime:

```

1  // Creiamo un caffè' semplice
2  Bevanda mioCaffe = new CaffeSemplice();
3
4  // Lo decoriamo con latte
5  mioCaffe = new AggiuntaLatte(mioCaffe);
6
7  // Lo decoriamo ancora con caramello
8  mioCaffe = new AggiuntaCaramello(mioCaffe);
9
10 System.out.println(mioCaffe.getDescrizione()); // Output: Caffè' espresso, con
11 latte, al caramello
12
13 System.out.println(mioCaffe.calcolaCosto()); // Output: 2.30

```

10.4 PATTERN COMPORTAMENTALI

10.4.1 Observer

Definisce una dipendenza 1:N tra oggetti; quando il soggetto cambia stato, tutti gli osservatori vengono notificati e aggiornati automaticamente. È il cuore del pattern MVC (Model-View-Controller).

Definiamo due ruoli:

- Il **Soggetto (Subject o Observable)**: colui che detiene lo stato. Mantiene una lista di chi è interessato ai suoi cambiamenti e fornisce metodi per iscriversi (attach) o disiscriversi (detach);
- Gli **Osservatori (Observers)**: coloro che vogliono essere avvisati. Implementano un'interfaccia comune con un metodo (spesso chiamato update()).

Quando lo stato del Soggetto cambia, lui semplicemente scorre la sua lista di iscritti e chiama il metodo update() su ciascuno di essi.

```

1  // 1. L'Interfaccia dell'Observer
2  public interface Observer {
3      void update(float temperatura, float umidita);
4  }
5
6  // 2. Il Soggetto
7  public class StazioneMeteo {
8      // La lista degli iscritti. Notare che dipende dall'interfaccia, non dalle
9      // classi concrete!
10     private List<Observer> iscritti = new ArrayList<>();
11     private float temperatura;
12     private float umidita;
13
14     public void iscrivi(Observer o) { iscritti.add(o); }
15     public void disiscrivi(Observer o) { iscritti.remove(o); }
16 }

```

```

17 // Il metodo magico
18 public void notificaIscritti() {
19     for (Observer o : iscritti) {
20         o.update(temperatura, umidita); // Notifica in modalita' "Push"
21     }
22 }
23
24 public void setMisurazioni(float temp, float umidita) {
25     this.temperatura = temp;
26     this.umidita = umidita;
27     notificaIscritti(); // Attiva a cascata l'aggiornamento
28 }
29 }

```

10.4.2 State

Permette a un oggetto di cambiare il proprio comportamento quando cambia il suo stato interno, modellando macchine a stati finiti tramite il polimorfismo.

Immagina di dover gestire la pubblicazione di un articolo in un CMS (Content Management System). Un articolo ha un ciclo di vita e può trovarsi in tre stati: Bozza, InRevisione e Pubblicato.

A seconda dello stato, il comportamento dei metodi cambia drasticamente:

- Se chiami pubblica() su una Bozza, l'articolo non viene pubblicato, ma passa InRevisione;
- Se chiami pubblica() su un articolo InRevisione (e sei un admin), passa a Pubblicato;
- Se chiami pubblica() su un articolo già Pubblicato, il sistema non fa nulla (o lancia un errore).

Senza il pattern State, ti ritroveresti a scrivere questo orrore dentro la classe Articolo:

```

1 // L'incubo della manutenibilita'
2 public void pubblica() {
3     if (statoAttuale == BOZZA) {
4         statoAttuale = IN_REVISIONE;
5     } else if (statoAttuale == IN_REVISIONE) {
6         if (utente.isAdmin()) {
7             statoAttuale = PUBBLICATO;
8         }
9     } else if (statoAttuale == PUBBLICATO) {
10        System.out.println("Gia' pubblicato!");
11    }
12 }

```

Se un domani aggiungi lo stato Archiviato, devi riaprire questa classe e modificare tutti i metodi (pubblica(), modifica(), cancella()), aggiungendo un nuovo if ovunque.

Il pattern State ribalta la situazione. Invece di usare una variabile (come una stringa o un Enum) per tracciare lo stato, **creiamo una classe per ogni stato possibile**. L'oggetto principale (il Context) delegherà ogni azione all'oggetto "Stato" attualmente attivo.

```

1 // 1. L'Interfaccia Comune
2 public interface StatoArticolo {
3     void pubblica(Articolo contesto);
4     void modifica(Articolo contesto);
5 }
6
7 // 2. Il Context (L'Articolo)
8 public class Articolo {
9     private StatoArticolo statoAttuale;
10
11     public Articolo() {
12         this.statoAttuale = new StatoBozza(); // Stato iniziale
13     }
14
15     public void setStato(StatoArticolo nuovoStato) {
16         this.statoAttuale = nuovoStato;
17     }
18
19     // L'Articolo non ha piu' "if". Delega semplicemente allo stato!
20     public void pubblica() {
21         statoAttuale.pubblica(this);
22     }
23 }
24
25 // 3. Gli Stati Concreti
26 public class StatoBozza implements StatoArticolo {
27     @Override
28     public void pubblica(Articolo contesto) {
29         System.out.println("L'articolo passa in revisione.");
30         contesto.setStato(new StatoInRevisione()); // Transizione di stato!
31     }
32
33     @Override
34     public void modifica(Articolo contesto) {
35         System.out.println("Modifica della bozza effettuata.");
36     }
37 }
38
39 public class StatoInRevisione implements StatoArticolo {
40     @Override
41     public void pubblica(Articolo contesto) {
42         System.out.println("L'articolo e' approvato e pubblicato.");
43         contesto.setStato(new StatoPubblicato());
44     }
45
46     @Override
47     public void modifica(Articolo contesto) {
48         System.out.println("Non puoi modificare un articolo in revisione!");
49     }
50 }

```

10.5 PATTERN MODERNI (NON PRESENTI IN GOF)

10.5.1 DependencyInjection

Si verifica quando le risorse da cui dipende un componente vengono fornite (**iniettate**) dall'esterno anziché essere istanziate dal componente stesso durante la sua esecuzione.

Il componente deve dichiarare (ad esempio tramite il proprio costruttore) il tipo (eventualmente generico) e la quantità di tutte le risorse da cui dipende.

Facciamo un semplice esempio. Immagina di dover scrivere un servizio che gestisce gli utenti e che deve salvare i dati in un database. L'approccio *naïve* (senza DI) è questo:

```
1 public class ServizioUtenti {
2     // Il servizio CREA la sua stessa dipendenza
3     private DatabaseMySQL database;
4
5     public ServizioUtenti() {
6         // ACCOPPIAMENTO FORTE: ServizioUtenti e' incollato a DatabaseMySQL
7         this.database = new DatabaseMySQL();
8     }
9
10    public void salvaUtente(Utente u) {
11        database.salva(u);
12    }
13 }
```

Nel DI una classe non dovrebbe mai creare (istanziare con `new`) le sue dipendenze complesse. Dovrebbe invece richiederle dall'esterno (questo prende il nome di **Inversion of Control**). Il codice diventa quindi:

```
1 // 1. Definisco un'astrazione (Interfaccia)
2 public interface Database {
3     void salva(Utente u);
4 }
5
6 // 2. Il servizio richiede l'astrazione, non l'implementazione concreta
7 public class ServizioUtenti {
8     private final Database database;
9
10    // INJECTION: La dipendenza viene fornita ("iniettata") dall'esterno
11    public ServizioUtenti(Database database) {
12        this.database = database;
13    }
14
15    public void salvaUtente(Utente u) {
16        database.salva(u);
17    }
18 }
```

CAPITOLO 11

Containerizzazione

Il settore del software è cambiato radicalmente. Siamo passati da software monolitici, caratterizzati da lunghi cicli di sviluppo e destinati a un singolo ambiente, a un approccio moderno. Oggi sviluppiamo servizi disaccoppiati, con miglioramenti rapidi e iterativi, che devono scalare velocemente e funzionare su molteplici ambienti.

Questa complessità ha generato quella che viene definita "**La Matrice Infernale**". Pensa a quanti stack tecnologici usiamo (linguaggi come Python o Java, framework, database come PostgreSQL o Redis). Ora incrocia questi stack con la molteplicità degli ambienti hardware: il laptop dello sviluppatore, i server di QA, il cloud pubblico o i server di disaster recovery. Far interagire tutto correttamente e migrare i servizi da un ambiente all'altro senza intoppi è un incubo logistico.

Il problema è stato risolto introducendo i **container standardizzati**:

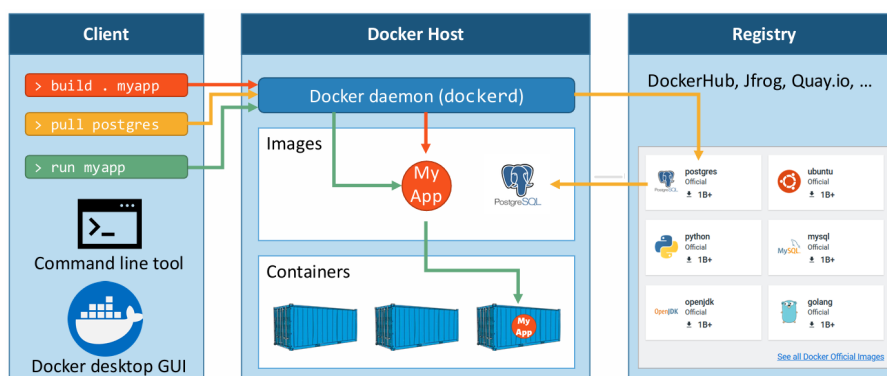
- Gli sviluppatori si preoccupano solo di cosa c'è *dentro* il container, semplificando il setup dell'ambiente ed evitando conflitti tra dipendenze e librerie;
- I sistemisti (Operations) si preoccupano solo di cosa c'è *fuori*, gestendo ogni container nello stesso identico modo per semplificarne il ciclo di vita.

Le macchine virtuali risolvono già questo problema, ma non in modo efficiente in quanto sono "pesanti" perché richiedono un intero sistema operativo ospite (Guest OS) separato per ogni applicazione. I container invece sono molto più leggeri perché poggiano su un Container Engine e condividono il sistema operativo (Operating System) della macchina ospitante.

11.1 DOCKER

Lo standard de facto per la containerizzazione del software è **Docker**. La sua architettura si basa su tre macro-componenti:

1. **Client**: lo strumento a riga di comando o GUI con cui si inviano le istruzioni;
2. **Docker Host**: la macchina su cui gira il demone Docker (dockerd), che gestisce materialmente le Immagini e i Container;
3. **Registry**: i repository (come DockerHub) da cui si possono scaricare (pull) immagini ufficiali o caricate da altri.



Per creare le immagini personalizzate scriviamo un **Dockerfile**, un semplice file di testo con una lista di istruzioni. I comandi fondamentali includono:

- **FROM:** per definire l'immagine di base (es. `FROM ubuntu:20.04` o `FROM python:3.8`);
- **RUN:** per eseguire comandi e installare le dipendenze;
- **COPY:** per copiare i file dal tuo computer all'interno del container;
- **EXPOSE:** per indicare su quali porte il container resterà in ascolto;
- **CMD:** il comando di default che verrà eseguito all'avvio del container.

11.2 SVILUPPO CLOUD-NATIVE E DOCKER-COMPOSE

Mettere un vecchio software in un container non risolve magicamente i problemi. Il software deve essere **container-native**, ovvero progettato fin dall'inizio trattando i container come unità di base dell'infrastruttura, al posto dei vecchi server fisici.

Poiché in un'app moderna i container devono comunicare tra loro, gestirli singolarmente da riga di comando diventa impossibile. Per questo utilizziamo **docker-compose**.

Tramite un file `.yaml` possiamo definire in un unico posto tutti i servizi che compongono la nostra applicazione, indicando per ciascuno quale Dockerfile utilizzare, la mappatura delle porte, le variabili d'ambiente necessarie per farli dialogare e le dipendenze di avvio.

CAPITOLO 12

Refactoring e Bad Smells

Il **refactoring** (inteso come sostantivo) è una modifica apportata alla struttura interna del software per renderlo più facile da capire e più economico da modificare. La condizione fondamentale e non negoziabile è che questa modifica avvenga senza cambiare il suo comportamento osservabile.

12.1 DEBITO TECNICO

Il debito tecnico si verifica quando il codice soddisfa i requisiti funzionali ma è implementato in modo subottimale, "sporco" o frettoloso.

Può essere:

- **Strategico/Intenzionale**: scelta deliberata di "rilasciare ora e gestire le conseguenze dopo". Esempi includono design non modulare, mancanza di scalabilità o implementazioni volutamente semplici/-complesse;
- **Non Strategico/Involontario**: risultato di processi imperfetti, come bug (memory leak), copertura dei test insufficiente, architetture fragili o "Bad Smells" accumulati casualmente.

Un debito eccessivo comporta imprevedibilità nella pianificazione, aumento dei bug, costi di correzione elevati e frustrazione dei programmatori. Strumenti di analisi della qualità come **SonarQube** possono fornire stime accurate del debito tecnico accumulato in un repository.

12.2 BAD SMELLS

I code smells sono indicazioni di problemi di design nel sistema. Sono stati classificati in cinque macro-categorie.

12.2.1 Bloaters (Elementi Gonfiati)

Codice, metodi e classi che hanno raggiunto proporzioni tali da essere difficili da gestire. Si accumulano solitamente nel tempo.

- **Long Method**: metodi troppo lunghi e difficili da capire;
- **Large Class**: classi che cercano di fare troppo, con troppe variabili di istanza;
- **Primitive Obsession**: riluttanza a usare piccole classi per concetti semplici (es. numeri di telefono, intervalli);

- **Long Parameter List:** liste di parametri lunghe, inconsistenti e difficili da usare;
- **Data Clumps:** gruppi di dati (3-4 elementi) che appaiono costantemente insieme.

12.2.2 Object-Orientation Abusers (Abusatori dell'Orientamento agli Oggetti)

Applicazione incompleta o errata dei principi della programmazione orientata agli oggetti.

- **Switch Statements:** uso eccessivo di switch invece del polimorfismo;
- **Refused Bequest:** una sottoclasse che non utilizza le proprietà o i metodi ereditati dalla superclasse;
- **Alternative Classes with Different Interfaces:** classi simili con interfacce diverse;
- **Temporary Field:** variabili di istanza impostate solo in circostanze specifiche.

12.2.3 Change Preventers (Ostacoli al Cambiamento)

Smell che rendono ogni modifica complicata e costosa, richiedendo interventi in molteplici punti.

- **Divergent Change:** una classe viene modificata spesso per ragioni diverse (violazione della separazione degli interessi);
- **Shotgun Surgery:** una singola modifica concettuale richiede piccoli interventi in molte classi diverse;
- **Parallel Inheritance Hierarchies:** ogni volta che si crea una sottoclasse in una gerarchia, è necessario crearne una anche in un'altra.

12.2.4 Dispensables (Elementi Superflui)

Elementi inutili la cui rimozione renderebbe il codice più pulito ed efficiente.

- **Duplicate Code:** stessa struttura di codice in più posti;
- **Lazy Class / Dead Code:** classi o porzioni di codice che non giustificano il costo del loro mantenimento;
- **Speculative Generality:** inserimento di "ganci" o casi speciali per esigenze future ipotetiche;
- **Comments:** spesso usati come "deodorante" per nascondere codice scritto male.

12.2.5 Couplers (Accoppiatori)

Smell che indicano un accoppiamento eccessivo tra classi.

- **Feature Envy:** Un metodo sembra più interessato ai dati di un'altra classe che a quelli della propria;
- **Message Chains:** lunghe catene di chiamate (es. `a.getB().getC().doSomething()`).
- **Middle Man:** una classe che delega gran parte del suo lavoro ad altre.

12.3 IL PROCESSO OPERATIVO DI REFACTORING

Il ciclo del refactoring si basa su queste fasi:

1. **Identificare lo smell:** individuare il problema peggiore (la parte più difficile);
2. **Selezionare il refactoring:** scegliere la tecnica adatta a quello specifico smell;
3. **Applicare il refactoring:** eseguire la trasformazione preservando il comportamento.

Le tecniche comuni per effettuare il refactoring sono:

Categoria	Esempi di tecniche
Refactoring di Classe	Aggiungi/Rimuovi sottoclasse, Estrai classe, Collapse Hierarchy
Refactoring di Metodo	Extract Method, Rename Method, Move Method, Push Up/Down Method
Refactoring di Attributo	Rename variable, Pull Up/Push Down variable, Create accessors
Gestione Parametri	Add Parameter, Replace Parameter with Method, Introduce Parameter Object

Esempi chiave:

- Extract Method: Se un frammento di codice può essere raggruppato, viene estratto in un nuovo metodo con un nome esplicativo.
- Push Method Down: Sposta un comportamento dalla superclasse a una sottoclasse se quel comportamento è rilevante solo per quest'ultima.
- Pull Up Method: Sposta metodi con risultati identici dalle sottoclassi alla superclasse per eliminare duplicazioni.

CAPITOLO 13

Principi SOLID

Prima di arrivare ai principi, bisogna prima di tutto riconoscere quali sono i **Software "Smells"**, ovvero i sintomi di un cattivo design:

- **Rigidità**: il sistema è difficile da modificare perché un singolo cambiamento costringe a modificare tante altre parti del codice (la cosiddetta "Shotgun Surgery");
- **Fragilità**: le modifiche causano rotture del sistema in punti imprevisti;
- **Immobilità**: il codice non è riutilizzabile;
- **Opacità**: è inutilmente complesso, difficile da capire e contiene ripetizioni.

Per combattere questi problemi sono stati introdotti i **5 Principi SOLID**. Analizziamoli uno per uno.

13.1 SRP: SINGLE RESPONSIBILITY PRINCIPLE (PRINCIPIO DI SINGOLA RESPONSABILITÀ)

Una classe deve avere una e **UNA SOLA** responsabilità, il che significa che deve avere un solo motivo per cambiare. Questo perché se una classe ha più responsabilità, queste si accoppiano; di conseguenza, una modifica a un requisito rende l'intera classe fragile.

Facciamo un esempio. Supponiamo di avere una classe `Report` che fa tre cose: ottiene i dati (`GetData()`), formatta il report (`FormatReport()`) e lo stampa (`Print()`).

```
1  class Report {
2      private IList<ReportDataElement> GetData(){
3          Console.WriteLine("\nGettingData...");
4          return new List<ReportDataElement>() // codice continua ma non visibile
5      }
6
7      private void FormatReport() {
8          Console.WriteLine("\nFormatting Report...");
9      }
10
11     public void Print() {
12         GetData();
13         FormatReport();
14         Console.WriteLine("\nPrinting Report...");
15     }
16 }
```

La soluzione corretta è dividere queste operazioni in quattro classi distinte: Report, ReportPrinter, ReportFormatter e DataAccess.

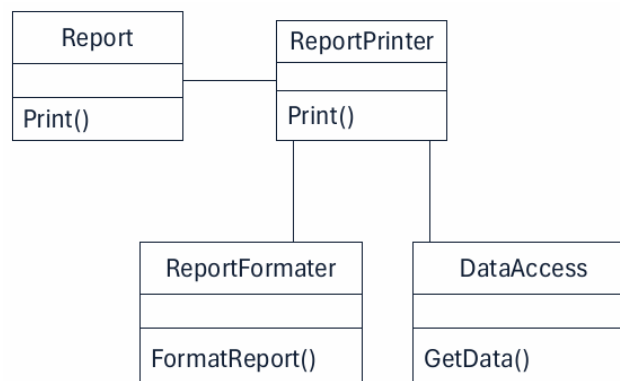
```
1 public class DataAccess {
2     public IList<ReportDataElement> GetData() {
3         Console.WriteLine("\nGetting Data...");
4         return new List<ReportDataElement>() // codice continua ma non visibile
5     }
6 }
```

```
1 public class ReportFormatter {
2     public void FormatReport() {
3         console.WriteLine("\nFormatting Report...");
4     }
5 }
```

```
1 public class ReportPrinter {
2     public void Print() {
3         DataAccess dataAccess = new DataAccess();
4         ReportFormatter reportFormatter = new ReportFormatter();
5         dataAccess.GetData();
6         reportFormatter.FormatReport();
7         Console.WriteLine("\nPrinting Report...");
8     }
9 }
```

```
1 class Report {
2     public void Print(){
3         ReportPrinter reportPrinter = new ReportPrinter();
4         reportPrinter.Print();
5     }
6 }
```

Questo porta codice più pulito, nomi delle classi più facili da scegliere, maggiore leggibilità e, soprattutto, facilità di riutilizzo.



13.2 OCP: OPEN/CLOSED PRINCIPLE

Le classi devono essere progettate in modo che il loro comportamento possa essere esteso senza modificare il codice sorgente esistente. Le chiavi per l'OCP sono l'astrazione e il polimorfismo dei sottotipi. Si ottiene introducendo classi base astratte o interfacce comuni.

Facciamo un esempio. Immaginiamo di dover cambiare formato di stampa passando al Tabloid o usando una stampante ad aghi. Invece di riempire il ReportPrinter di istruzioni if, si creano sottoclassi specializzate come TabloidReportFormatter e TabloidReportPrinter che ereditano dalle classi base, sovrascrivendo i metodi necessari.

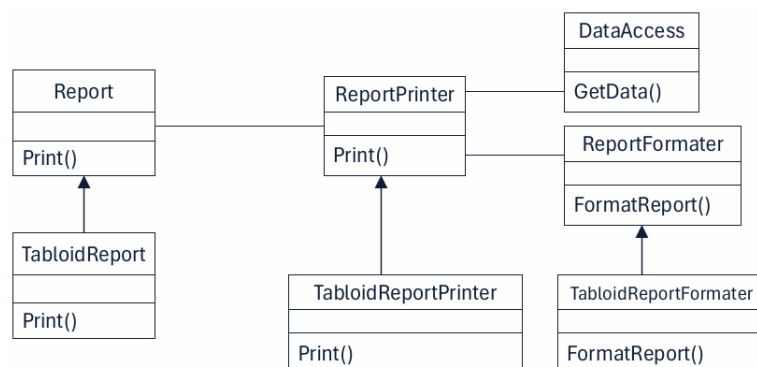
Al codice precedente aggiungeremo:

```
1 public class TabloidReportFormatter : ReportFormatter {
2     public override void FormatReport() {
3         Console.WriteLine("\nFormatting Report for 11 x 17...");
4     }
5 }

1 public class TabloidReportPrinter : ReportPrinter {
2     public override void Print() {
3         DataAccess dataAccess = new DataAccess();
4         ReportFormatter reportFormatter = new TabloidReportFormatter();
5         dataAccess.GetData();
6         reportFormatter.FormatReport();
7         Console.WriteLine("\nPrinting Report to dot-matrix printer...");
8     }
9 }

1 public class TabloidReport : Report {
2     public override void Print() {
3         ReportPrinter reportPrinter = new TabloidReportPrinter();
4         reportPrinter.Print();
5     }
6 }
```

Questo porta un design più stabile ed estensibile, modifiche isolate, codice più facile da testare e da sostituire.



13.3 LSP: LISKOV SUBSTITUTION PRINCIPLE

Se *S* è un sottotipo di *T*, allora oggetti dichiarati in un programma di tipo *T* possono essere sostituiti con oggetti di tipo *S* senza alterare la correttezza dei risultati del programma.

Il problema sta nel fatto che l'ereditarietà definisce una relazione "is a" (è un), ma è facile farsi prendere la mano e creare design sbagliati.

Supponiamo di avere le seguenti classi:

```
1 public class Rectangle {
2     private double height;
3     private double width;
4
5     public void setHeight(double h) {
6         height = h;
7     }
8
9     public void setHeight(double w) {
10        width = w;
11    }
12 }
```

```
1 public class Square extends Rectangle {
2     public void setHeight(double h) {
3         super.setHeight(h);
4         super.setWidth(h);
5     }
6
7     public void setWidth(double h) {
8         super.setHeight(h);
9         super.setWidth(h);
10    }
11 }
```

Una classe *Square* che eredita da *Rectangle* può violare l'LSP se i metodi per impostare altezza e larghezza alterano forzatamente entrambi i lati, rompendo le aspettative di chi usa la classe base *Rectangle*.

Un altro errore comune è ereditare una classe *Stack* da una classe *List* solo perché i metodi della lista sono "già implementati". È un'ereditarietà di implementazione scorretta.

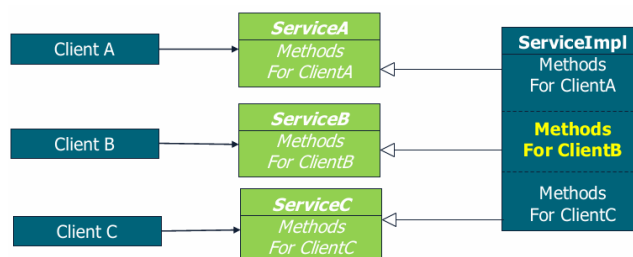
Se non si rispetta l'LSP, le gerarchie diventano un disastro e i test di unità falliscono. Inoltre, violare questo principio porta spesso a violare anche l'OCP.

13.4 ISP: INTERFACE SEGREGATION PRINCIPLE

I client non devono essere forzati a dipendere da interfacce che non utilizzano. È preferibile avere molte interfacce specifiche per il client piuttosto che un'unica interfaccia generica.

Interfacce con troppi metodi creano accoppiamenti involontari. Se un'interfaccia *IVehicle* ha il metodo *openDoors()*, una classe *MotorBike* che la implementa sarà costretta a fornire un'implementazione per un metodo che non le serve.

Un buon design è rappresentato in questo modo:

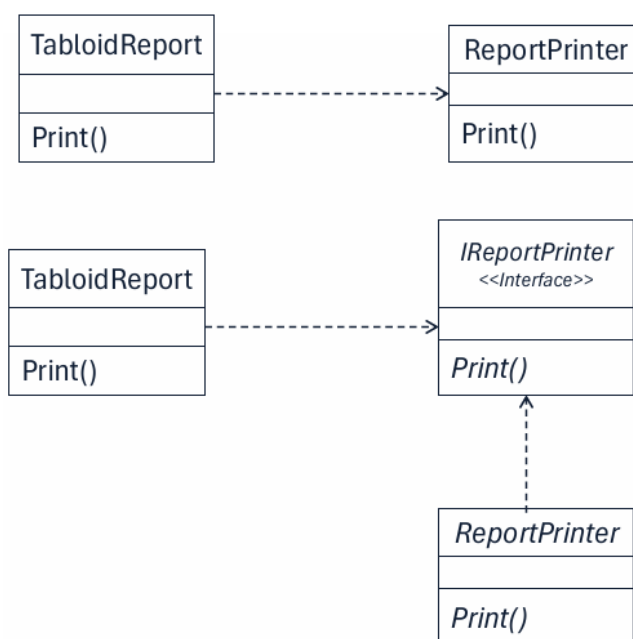


L'ISP porta aumento della coesione, design stabile e supporto diretto al Principio di Liskov.

13.5 DIP: DEPENDENCY INVERSION PRINCIPLE

I moduli di alto livello (che contengono la logica di business) non devono dipendere dai moduli di basso livello (che contengono i dettagli di implementazione). Entrambi devono dipendere dalle astrazioni. Le astrazioni non devono dipendere dai dettagli. Lo si ottiene accedendo alle istanze tramite interfacce o classi astratte.

Consideriamo l'esempio precedente. Invece di far dipendere la classe base direttamente da una stampante concreta, si introduce un'interfaccia `IReportPrinter`. In questo modo, la classe `TabloidReport` utilizzerà l'interfaccia, invertendo così la direzione della dipendenza.



CAPITOLO 14

Testing

Secondo lo standard IEEE 29119, il testing è definito come l'insieme di attività condotte per facilitare la scoperta e/o la valutazione delle proprietà di uno o più elementi di test. Bisogna avere ben chiara la differenza tra due concetti cruciali:

- **Verification (Verifica):** *stiamo costruendo il prodotto nel modo giusto?*. Serve a determinare se i prodotti di una fase di sviluppo soddisfano i requisiti stabiliti nella fase precedente;
- **Validation (Validazione):** *stiamo costruendo il prodotto giusto?*. È il processo di valutazione del software alla fine dello sviluppo per assicurare la conformità con l'uso previsto.

È altrettanto importante usare la terminologia corretta. I seguenti termini, infatti, non sono sinonimi:

- **Error (Errore):** è l'errore umano nella programmazione, dovuto a incomprensioni, mancanza di conoscenza o distrazione;
- **Fault (Difetto o Bug):** è l'introduzione dell'errore nel codice del software.
- **Failure (Malfunzionamento):** è l'esecuzione del software che non fornisce il risultato atteso. Avviene dinamicamente e può essere osservata solo eseguendo il programma.

Esempio:

```
1  int raddoppia(int x) {  
2      int y;  
3      y = x*x;  
4      return (y);  
5  }
```

Per il valore di ingresso $x = 3$ si ha il valore di uscita $y = 9$. La causa di questa **failure** è il **fault** di linea 2, in cui al posto dell'operatore $+$ è usato l'operatore $*$.

Non tutti i fault generano failure. Una failure può essere generata da più fault. Usiamo il termine **defect** quando non è importante distinguere fra fault e failure, per riferirsi sia alla causa (fault) che all'effetto (failure).

14.1 MODELLI PER MIGLIORARE L'AFFIDABILITÀ DEL SOFTWARE

- **Maturity-Fault Avoidance:** si previene l'introduzione di difetti nel sistema prima che sia realizzato;
- **Fault Detection:** si identificano gli stati di errore e trovare il difetto prima di rilasciare il sistema. Si individuano attraverso:
 - Testing: esecuzioni mirate con input specifici;
 - Debugging: esperimenti non controllati.
- **Fault Tolerance:** si assume che un sistema possa contenere bug e che i fallimenti del sistema possano essere gestiti a runtime.

14.2 SPECIFICA ED ESECUZIONE DEI TEST

Come si struttura un test in modo professionale? Un "*Test case specification*" richiede un set minimo di informazioni:

1. **ID e Descrizione;**
2. **Preconditions (Precondizioni):** ipotesi che devono essere verificate prima dell'esecuzione del test (es. il database deve essere attivo);
3. **Valori di Input;**
4. **Expected Output (Risultato Atteso):** chiamato anche l'Oracolo;
5. **Expected Postconditions (Postcondizioni):** ipotesi che devono essere verificate dopo l'esecuzione.

Durante l'esecuzione, si genera un report che aggiunge l'**Output effettivo** e il **Risultato** finale. Il risultato può essere:

- **Successo:** pre e post-condizioni vere, e l'output effettivo equivale a quello atteso;
- **Fallimento:** pre e post-condizioni vere, ma l'output effettivo è diverso da quello atteso;
- **Non Applicabile (N/A):** se una precondizione o postcondizione non è vera.

L'**Oracolo** è il meccanismo o la persona che sa esattamente quale debba essere l'output atteso. Può essere umano, un altro software (software oracle), implicito (es. test per evitare crash) o derivato da specifiche formali.

I livelli di testing sono:

- **Unit Testing:** verifica di singoli componenti (metodi o funzioni). Assicura che ogni unità sia implementata correttamente;
- **Integration Testing:** testa l'interazione tra componenti o moduli integrati. Assicura che i moduli comunichino correttamente;
- **Functional Testing:** trova le differenze tra use case model e il sistema;
- **System Testing:** trova le differenze tra requisiti non funzionali e il sistema.

14.3 QUALITÀ E MISURAZIONE (LA COPERTURA)

Un buon test deve possedere specifici requisiti di qualità: Accuratezza, Ripetibilità, Robustezza, Capacità di localizzare i guasti, Tracciabilità (rispetto ai requisiti), Efficacia ed Efficienza.

Dato che non conosciamo a priori il numero di fault presenti in un sistema, valutare l'efficacia reale è difficile. Usiamo quindi misure approssimate, la principale è la **Code Coverage** (Copertura del codice).

Se una porzione di codice (LOC, metodi o classi) contiene un fault ma non viene mai eseguita dai test, il difetto non verrà mai trovato. Maggiore è la quantità di codice che eseguiamo, maggiore è la "speranza" di trovare difetti.

14.4 TECNICHE BLACK BOX (TEST FUNZIONALE)

Il **testing Black Box** si concentra sui requisiti funzionali senza considerare la struttura interna del codice, trattando l'applicazione come una scatola nera. È ideale per verificare che il software soddisfi i requisiti definiti.

Le caratteristiche principali sono:

- Non richiede conoscenza del codice;
- Si applica a livello di unità, integrazione e sistema;
- Non rileva errori derivanti da funzionalità extra non richieste.

Le tecniche principali si basano sul partizionamento del dominio di input:

- **Classi di Equivalenza:** l'idea alla base di questa tecnica è dividere i valori dei campi di input in *classi*. Facciamo un'assunzione forte: ipotizziamo che il programma si comporti esattamente allo stesso modo per ogni valore appartenente alla stessa classe. Queste classi formano una *partizione* del dominio: la loro unione copre tutti gli input possibili e non ci sono intersezioni tra di esse. Per ogni input dobbiamo individuare:
 - Classi Valide: insiemi di valori considerati accettabili dal sistema;
 - Classi Invalide: insiemi di valori che innescano condizioni di errore (es. valori fuori range).
- **Valori limite:** siccome gli errori si annidano spesso ai confini dei domini di input, analizziamo i valori esatti del limite, e i valori immediatamente al di sotto e al di sopra di esso. Ad esempio, per i giorni di un mese (1-31), testeremo 0, 1, 2, un valore medio (es. 15), 30, 31, 32.

I criteri di copertura qui si dividono in:

- **Each Choice Coverage (ECC):** copertura minima. Ogni classe di equivalenza è coperta da almeno un test case;
- **All Combinations Coverage (ACC):** più esaustiva. Si testano tutte le combinazioni possibili delle classi di equivalenza.

14.4.1 Esempio

Immaginiamo di dover testare un metodo che riceve in input una data formata da tre parametri: `Giorno` (intero), `Mese` (stringa) e `Anno` (intero). Il metodo deve restituire un booleano (vero se la data è valida) e una stringa (il giorno della settimana, oppure "Errore"). Sappiamo che il calendario gregoriano è entrato in vigore nel 1582.

Ci sono due approcci per definire le classi di equivalenza per questo problema. Iniziamo con l'approccio basato sull'Interfaccia.

Qui consideriamo i parametri di input in totale isolamento, guardando solo al loro tipo e range di base.

- **Giorno (range 1-31)**
 - Valida: `1 <= giorno <= 31`
 - Invalide: `giorno <= 0`, `giorno > 31`
- **Mese (12 stringhe previste)**
 - Valida: mese appartenente all'insieme {gennaio, ..., dicembre}
 - Invalida: mese non appartenente a quell'insieme
- **Anno (dal 1582 in poi)**
 - Valida: `anno >= 1582`
 - Invalida: `anno < 1582`

L'altro approccio è quello basato sulle funzionalità. Qui modelliamo le classi rispetto al comportamento atteso, tenendo conto delle relazioni tra i parametri dettate dalla logica del dominio (es. gli anni bisestili e la durata dei mesi).

- **Giorno (6 classi):** `giorno <= 0`, `1 <= giorno <= 28`, `giorno = 29`, `giorno = 30`, `giorno = 31`, `giorno > 31`
- **Mese (4 classi):** mesi di 31 giorni, mesi di 30 giorni, Febbraio, mesi non validi
- **Anno (3 classi):** `anno < 1582`, `anno = 1582` (anno critico), `anno > 1582`

Nota: al variare dell'approccio, cambia drasticamente il numero di test che andrai a generare! L'approccio funzionale è molto più robusto per trovare difetti logici.

Una volta identificate le classi (supponiamo di usare l'approccio basato sull'interfaccia con 3 classi per il Giorno, 2 per il Mese, 2 per l'Anno), come combiniamo i valori per creare i test case? Usiamo i criteri di copertura.

Partiamo con il criterio Each Choice Coverage (ECC). Richiede che **ogni singola classe di equivalenza (di ogni input) sia coperta da almeno un test case**.

Per calcolare il numero minimo di test case necessari, basta guardare quale input ha il maggior numero di classi. Nel nostro caso base (`Giorno=3`, `Mese=2`, `Anno=2`), ci basterebbero 3 test case per coprire tutto.

Aggiungendo una classe invalida per il tipo di dato errato (es. inserire una stringa in un campo intero), arriviamo a 4 test case:

Test Case	Giorno (Classe)	Mese (Classe)	Anno (Classe)
TC1	1 (<i>Valida</i>)	Gennaio (<i>Valida</i>)	1980 (<i>Valida</i>)
TC2	0 (<i>Invalida</i>)	Brumaio (<i>Invalida</i>)	1492 (<i>Invalida</i>)
TC3	35 (<i>Invalida</i>)	Gennaio (<i>Valida</i>)	1980 (<i>Valida</i>)
TC4	"primo" (<i>Invalida</i>)	Gennaio (<i>Valida</i>)	"duemila" (<i>Invalida</i>)

Passiamo ora all'approccio All Combinations Coverage (ACC). Richiede la copertura di **tutte le combinazioni possibili delle classi di equivalenza**. Il numero di test case sarà pari al prodotto delle cardinalità delle classi di ogni input.

Riprendendo le classi base dell'approccio a interfaccia (Giorno=3 classi, Mese=2 classi, Anno=2 classi), il numero di test case per l'ACC sarà $3 \cdot 2 \cdot 2 = 12$ test case.

Bisognerebbe andare ad incrociare tutto:

- Giorno Valido + Mese Valido + Anno Valido
- Giorno Invalido(≤ 0) + Mese Valido + Anno Valido
- Giorno Invalido(> 31) + Mese Valido + Anno Valido
- ...

Se usassimo l'ACC sull'approccio funzionale (6 classi Giorno · 4 classi Mese · 3 classi Anno), avremmo bisogno di ben 72 test case.

14.5 TECNICHE WHITE BOX (TEST STRUTTURALE)

Il **testing White Box** analizza la struttura interna del codice per verificare che tutte le possibili strade logiche siano coperte.

Le caratteristiche principali sono:

- Si basa su grafi del flusso di controllo (CFG) che rappresentano i percorsi logici del codice;
- Individua errori legati alla struttura del programma;
- Non rileva mancate implementazioni di requisiti.

14.5.1 Grafo CFG

Il Grafo del Flusso di controllo di un programma P è una quadrupla $CFG(P) = (N, E, n_i, n_f)$, dove (N, E) è un grafo diretto con archi etichettati

$$n_i \in N, \quad n_f \in N, \quad N \setminus \{n_i, n_f\} = N_s \cup N_p$$

- n_i è il nodo iniziale
- n_f è il nodo finale
- N_s è l'insieme dei nodi che rappresentano le istruzioni

- N_p è l'insieme dei nodi che rappresentano i predicati (istruzioni come if, while, ...)

$$E \subseteq (N \setminus n_f) \times (N \setminus n_i) \times (\text{true}, \text{false}, \text{uncond})$$

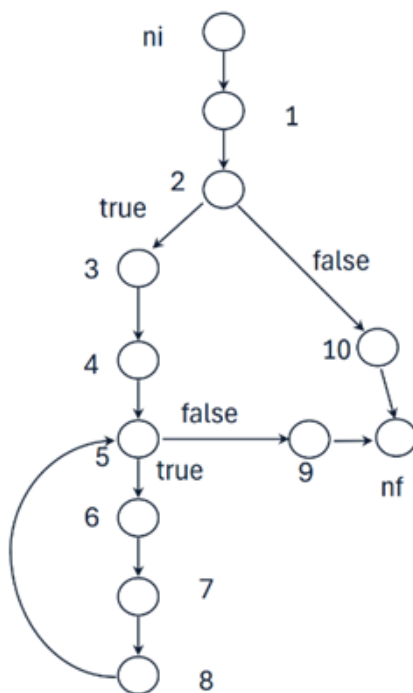
- Un nodo in $N_s \cup \{n_i\}$ ha un solo successore ed il suo arco è etichettato con **uncond**
- Un nodo in N_p ha due successori immediati e i suoi archi uscenti sono etichettati rispettivamente con **true** e **false**
- Un $CFG(P)$ è **ben formato** se esiste un cammino dal nodo iniziale n_i ad ogni nodo in $N \setminus \{n_i\}$ e da ogni nodo in $N \setminus \{n_f\}$ al nodo finale n_f
- Diremo semplicemente **cammino** o cammino totale un cammino da n_i a n_f

Esempio di CFG:

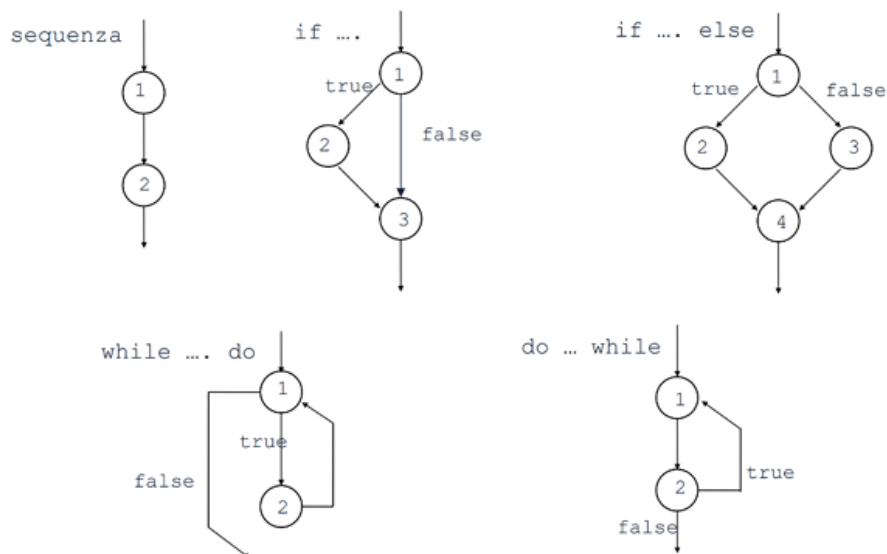
```

1  int foo (int x) {
2      int y,n;
3      if (x>0) {
4          n = 1;
5          y = 1;
6          while (x>1) {
7              n = n + 2;
8              y = y + n;
9              x = x - 1;
10         }
11         return y;
12     }
13     return x;
14 }

```

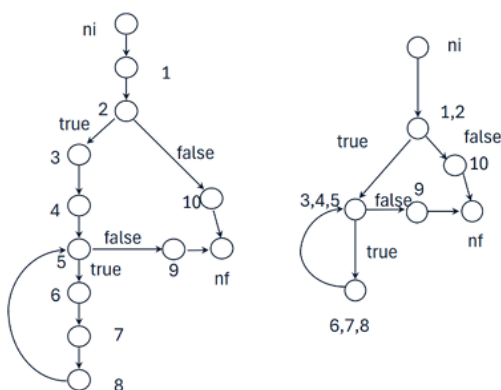


Costruzione del control flow graph per programmi strutturati:



Sequenze di nodi possono essere collassate in un solo nodo, purché nel grafo semplificato vengano mantenuti tutti i branch (punti di decisione e biforcazione del flusso di controllo). Tale nodo collassato può essere etichettato con i numeri dei nodi in esso ridotti.

```
int foo(int x) {
1 int y,n;
2 if (x>0) {
3     n = 1;
4     y = 1;
5     while (x>1) {
6         n = n + 2;
7         y = y + n;
8         x = x - 1;
9     }
10 return y;
}
return 0;
}
```



14.5.2 Copertura

La **copertura** rappresenta la percentuale di un determinato elemento del codice (nodi, decisioni, cammini, ecc...) che è stato effettivamente eseguito durante i test rispetto al totale degli elementi esistenti nel programma.

$$\text{Copertura}(\%) = \left(\frac{\text{Elementi eseguiti}}{\text{Elementi totali}} \right) \cdot 100$$

La copertura può riguardare diversi aspetti del codice o del flusso di controllo. Ecco i tipi principali:

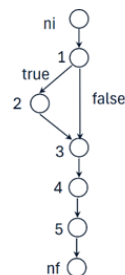
Copertura dei Nodi (Node Coverage)

Verifica che ogni nodo del CFG sia stato raggiunto ed eseguito almeno una volta durante l'esecuzione dei test. (TER = test effectiveness ratio)

$$TER = \frac{\text{n.ro di statement eseguiti}}{\text{n.ro di statement totale}}$$

Esempio:

```
void statement(double x, double y)
{
1.  if (x != 0)
2.      x = x + 10;
3.  y = y/x;
4.  cout << x;
5.  cout << y;
}
```



I nodi 1,3,4,5 vengono sempre eseguiti. Il "collo di bottiglia" è il nodo 2. Per coprire tutti i nodi non ci basta altro che prendere un qualunque $x \neq 0$. Il valore di y non è rilevante per la copertura del flusso. Quindi, se prendiamo $x = 5$ e $y = 100$, il percorso sarà $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$. **Risultato: Node Coverage = 100%.**

Esistono però casi di failure:

1. **Caso** $x = 0$

- Cosa succede? Al Nodo 1 la condizione $0 \neq 0$ è Falsa;
- Il flusso prende l'arco false e va direttamente al Nodo 3;
- Al Nodo 3 viene calcolato $y = y / 0$. **Crash.** Eccezione generata.

2. **Caso** $x = -10$

- Cosa succede? Al Nodo 1 la condizione $-10 \neq 0$ è Vera;
- Il flusso prende l'arco true ed entra nel Nodo 2;
- Al Nodo 2 viene calcolato $x = -10 + 10$, quindi x diventa 0;
- Al Nodo 3 viene calcolato $y = y / 0$. **Crash.** Eccezione generata.

Da questo possiamo capire che il Node Coverage è una condizione necessaria, ma assolutamente non sufficiente per garantire l'assenza di bug critici.

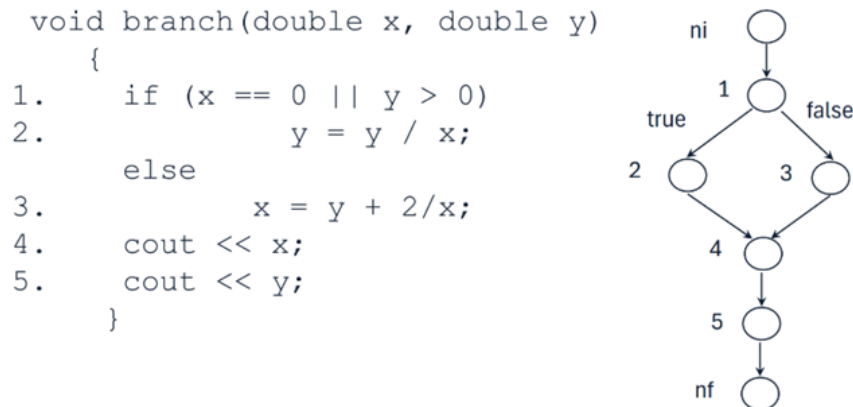
Copertura delle Decisioni (Branch)

La copertura delle decisioni verifica che ogni decisione logica (nodo predicato) nel CFG sia stata valutata sia come vera sia come falsa almeno una volta.

$$TER = \frac{\text{n.ro di branch eseguiti}}{\text{n.ro di branch totali}}$$

Nota: la copertura delle decisioni implica la copertura dei nodi.

Esempio:



Per attraversare questo grafo dall'inizio alla fine abbiamo solo due percorsi logici:

1. $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ (passando per il ramo Vero)
2. $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ (passando per il ramo Falso)

Il criterio della Branch Coverage (o Decision Coverage) richiede che ogni arco (edge) del CFG sia incluso in almeno un caso di test. Questo significa che dobbiamo scrivere almeno due casi di test: uno che faccia valutare la decisione come Vera e uno come Falsa.

- Per il ramo Vero (nodo 2): la condizione deve essere $x == 0 \ || \ y > 0$
- Per il ramo Falso (nodo 3): la condizione deve essere rigorosamente l'opposto logico. Per le leggi di De Morgan, il negato di $(x == 0 \ || \ y > 0)$ diventa $x != 0 \ \&\& \ y <= 0$.

Vogliamo ottenere il 100% di Branch Coverage, quale input scegliamo?

- **Test 1 (ramo vero):** guardiamo la condizione $x == 0 \ || \ y > 0$. Basta che y sia maggiore di 0 per renderla vera. Scegliamo quindi $x = 5$, $y = 10$. Il flusso va al Nodo 2. Viene eseguito $y = 10/5$. Nessun problema.
- **Test 2 (ramo falso):** guardiamo la condizione $x != 0 \ \&\& \ y <= 0$. Scegliamo $x = 5$, $y = -10$. Il flusso va al Nodo 3. Viene eseguito $x = -10 + 2/5$. Nessun problema.

Entrambi i casi sono stati coperti! Ma attenzione, esiste invece un caso di failure, ovvero quando $x = 0$. Vediamo:

- Se $x = 0$, la condizione $0 == 0 \ || \ y > 0$ è vera.
- Il flusso va al Nodo 2.
- Esegue $y = y/0$. **CRASH. Divisione per 0.**

Perché la nostra suite di test perfetta al 100% di Branch Coverage non ha trovato questo bug catastrofico?

Il problema è che la Branch Coverage testa solo la Decisione nella sua interezza (il risultato finale Vero o Falso). Non ti obbliga a testare le singole Condizioni logiche che la compongono. Nel nostro Test 1, abbiamo reso Vera la decisione sfruttando solo la clausola $y > 0$. Abbiamo completamente ignorato l'impatto della clausola $x == 0$.

Per scovare questo bug con certezza, avremmo dovuto usare la **Condition/Decision Coverage**.

Copertura dei Nodi (Node Coverage)

Questo criterio estende la copertura delle decisioni includendo anche le condizioni atomiche che compongono una decisione logica complessa.

14.6 JUNIT

L'Unit testing riguarda la porzione più piccola del software isolabile (metodi, classi). Dovrebbe essere progettato ed eseguito contemporaneamente allo sviluppo stesso.

Vantaggi

- Lo sviluppatore conosce esattamente le responsabilità della classe che ha sviluppato e i risultati che da essa si attende.
- Lo sviluppatore conosce esattamente come si accede alla classe, ad esempio:
 - Quali precondizioni devono essere poste prima di poter eseguire un caso di test;
 - Quali postcondizioni sui valori dello stato degli oggetti devono verificarsi.

Svantaggi

- Lo sviluppatore tende a difendere il suo lavoro ... troverà meno errori di quanto possa fare un tester!

L'automatizzazione dell'esecuzione dei casi di test porta innumerevoli vantaggi:

- **Tempo risparmiato** (nell'esecuzione dei test).
- **Affidabilità dei test** (non c'è rischio di errore umano nell'esecuzione dei test)
- **Riuso (parziale)** dei test a seguito di modifiche nella classe.

JUnit è un framework di unit testing per il linguaggio di programmazione Java. Esso include una classe di test detta `TestRunner` che:

- Funge da main (può essere chiamata dall'esterno).
- Esplora automaticamente le classi di test individuando tutti i loro component.
- Li esegue secondo un preciso schema.
- Genera i risultati dei test.

14.6.1 Componenti di un test JUnit 4

Una classe di test contiene:

- Un metodo detto di **setup**, preceduto dall'annotazione `Before` che viene eseguito prima dell'esecuzione di ogni test
 - Utile per settare precondizioni comuni a più di un caso di test.
 - Nel metodo di setup bisogna anche verificare le precondizioni.
- Un metodo detto di **teardown**, preceduto dall'annotazione `After` che viene eseguito dopo ogni caso di test.
 - Utile per resettare le postcondizioni.
 - Nel metodo di teardown bisogna anche verificare le eventuali postcondizioni.
- Un metodo per ogni caso di test, ognuno preceduto dall'annotazione `Test`
 - In ogni metodo di test bisogna verificare che il risultato del test sia quello atteso.

Nel metodo di test si usano le **Asserzioni** per verificare i risultati (es. `assertEquals(valoreAtteso, valoreEffettivo)`, `assertNull()`, `assertTrue()`). Se un'asserzione è falsa, il test fallisce ("Test Failed").

Esempio:

```
1 package mioProgetto;
2 import static org.junit.Assert.*;
3 import org.junit.Test;
4
5 public class miaClasseTest {
6     @Test
7     public void primoTestCase() {
8         ...Codice
9         assertEquals(messaggio, valoreAtteso, valoreDaEsaminare);
10    }
11
12    @Test
13    public void secondoTestCase() {
14        ...Codice
15        assertNull(valoreCheDeveEssereNulla);
16    }
17    ...
18 }
```

`import static org.junit.Assert.*;` serve per importare metodi (statici) e annotazioni del framework JUnit. `Test` è un'annotazione per marcare i metodi che si considerano di Test. Non è (più) necessario ma è buona norma usare 'test' come prefisso del nome dei metodi di test (obbligatorio in JUnit 3).

```
1 @Test
2 public void testMetodoDaTestare() {
3     ...
4 }
```

14.6.2 Asserzioni

È un'affermazione che può essere vera o falsa. Se l'asserzione è

- **Vera** $\Rightarrow$ il test è passato senza rilevare malfunzionamenti (**Test Passed**)
- **Falsa** $\Rightarrow$ il test è fallito ed il codice testato non si comporta come atteso, quindi c'è un errore a tempo dinamico (**Test Failed**).

Le asserzioni sono utilizzate sia per verificare gli oracoli che le postcondizioni. Potrebbero essere utilizzate anche per verificare le precondizioni: se la precondizione fallisce, il test non viene proprio eseguito (e viene riportato come fallito).

14.6.3 assertEquals()

La sintassi è

```
1 assertEquals(Object expected, Object actual)
```

Va a buon fine se e solo se `expected.equals(actual)` restituisce **True**.

- `expected` è il valore atteso.
- `actual` è il valore effettivamente rilevato.

Una variante è

```
1 assertEquals(String message, Object expected, Object actual)
```

Qui si specifica un messaggio che il runner stampa in caso di fallimento dell'asserzione: molto utile per localizzare immediatamente l'asserzione che causa il fallimento di un test-case ed avere i primi messaggi diagnostici.

Esempio di utilizzo:

```
1 public void testMetodoDaTestare() {  
2     miaClasse mioOggetto = new miaClasse("par1");  
3     assertEquals("par1", mioOggetto.getNome());  
4 }
```

14.6.4 Classi di test

Le classi di test possono essere create nella cartella `test/java` esattamente come si possono creare le classi normali. Potenzialmente le classi possono trovarsi anche in altri folder, eventualmente anche nello stesso folder delle classi di sviluppo. La separazione tra classi di sviluppo e classi di test ha effetti benefici nella divisione dei compiti di lavoro e nella compilazione ed esecuzione separata delle varie parti.

Vediamo un esempio. Supponiamo di avere una classe per la gestione del carrello di un sito di E-Commerce. Il nome della classe è `Cart`. I metodi implementati sono:

- `getTotal()` - restituisce il numero di elementi nel carrello.
- `insertProduct(String nome)` - inserisce il prodotto nel carrello se c'è ancora spazio (il carrello può contenere al massimo tre prodotti) e, nel caso, incrementa di uno il numero di elementi nel carrello.
- `removeProduct(String nome)` - se il prodotto indicato è nel carrello lo elimina e decrementa di uno il numero di elementi nel carrello.
- `removeAllProducts()` - rimuove tutti i prodotti e azzera il numero di elementi nel carrello.

- isEmpty() – restituisce true se il carrello è vuoto.

```

1 import java.util.ArrayList;
2
3 public class Cart {
4     private int total;
5     private ArrayList<String> product;
6
7     public Cart() {
8         product = new ArrayList<String>();
9         total = 0;
10    }
11
12    public int getTotal(){return total;}
13
14    public void insertProduct(String nome) {
15        if (total<3) {
16            product.add(nome);
17            total++;
18        }
19    }
20
21    public void removeProduct(String nome) {
22        if (product.contains(nome)) {
23            product.remove(nome);
24            total--;
25        }
26    }
27
28    public void removeAllProducts() {
29        product.clear();
30        total = 0;
31    }
32
33    public boolean isEmpty() {return total==0;}
34 }

```

Scriviamo ora i casi di test. L'obiettivo è eseguire tutti i metodi e tutte le righe di codice. Prima di tutto abbiamo bisogno di un oggetto Cart. Siccome ogni test avrà bisogno di un carrello valido, allora ha senso che lo inizializziamo nel metodo setup e lo distruggiamo nel metodo tearDown.

```

1 import org.junit.After;
2 import org.junit.Before;
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class CartTest {
7     private Cart cart;
8
9     @Before
10    public void setUp() throws Exception {
11        cart = new Cart();
12        assertFalse(cart == null);
13    }
14
15    @After
16    public void tearDown() throws Exception {
17        cart = null;
18        assertTrue(cart==null);
19    }
20 }
21

```

Con il primo test vogliamo provare ad inserire un elemento nel carrello e controllare che il totale sia stato effettivamente incrementato.

```
1  @Test
2  public void testInsertAProduct() {
3      cart.insertProduct("prodotto1");
4      assertEquals("Errore nel totale", 1, cart.getTotal());
5  }
```

Altri test possono essere:

```
1  // controlla che cart nasca vuoto
2  @Test
3  public void testEmptyCart() {
4      assertTrue("Doveva essere vuoto", cart.isEmpty());
5  }
6
7  //inserisce, poi rimuove
8  @Test
9  public void testRemoveOneProduct() {
10     cart.insertProduct("prodotto1");
11     cart.removeProduct("prodotto1");
12     assertEquals("Doveva essere rimosso", 0, cart.getTotal());
13 }
```

14.6.5 Copertura

Quanta parte del codice è stata coperta dai test effettuati? C'è una complessa teoria dietro questa definizione ma è molto semplice avere una prima valutazione. IntelliJ si integra con uno strumento di calcolo della copertura (JaCoCo) che può essere avviato con l'opzione **Run Test with Coverage...**

L'osservazione del codice non eseguito dai test ci può portare alla scrittura di nuovi casi di test che possano portare a eseguire tutto il codice disponibile. Nel nostro esempio ci rimangono da eseguire il caso di carrello pieno, l'eliminazione di un prodotto non presente nel carrello e l'eliminazione di tutti i prodotti dal carrello.

```
1  @Test
2  public void testRemoveAllProducts() {
3      cart.insertProduct("prodotto1");
4      cart.insertProduct("prodotto2");
5      cart.removeAllProducts();
6      assertTrue("Doveva essere vuoto", cart.isEmpty());
7  }
8
9  @Test
10 public void testRemoveProductNotInCart() {
11     cart.insertProduct("prodotto1");
12     cart.removeProduct("prodotto2");
13     assertEquals("Non doveva essere rimosso", 1, cart.getTotal());
14 }
15
16 @Test
17 public void testInsertMultipleProducts() {
18     cart.insertProduct("prodotto1");
19     cart.insertProduct("prodotto1");
20     cart.removeProduct("prodotto1");
21     assertEquals("Errore nel totale", 1, cart.getTotal());
22 }
23
24 @Test
25 public void testInsertMoreThanThreeProducts() {
26     cart.insertProduct("prodotto1");
27     cart.insertProduct("prodotto2");
```

```

28     cart.insertProduct("prodotto3");
29     cart.insertProduct("prodotto4");
30     assertEquals("Errore nel totale", 3, cart.getTotal());
31 }

```

14.7 ISOLATION TESTING AND INTEGRATION TESTING

I test unitari possono essere applicati solo a parti di software completamente isolate:

- Isolate dal resto del software (altre classi, librerie, ecc.)
- Isolate da altri software (sistema operativo, ecc.)
- Isolate da altre risorse (database, file, rete, ecc.)

Tuttavia, nella realtà non si può testare un modulo in isolamento se questo comunica con un database, con la rete, con altri moduli non ancora testati, oppure se modifica file e non può essere eseguito in parallelo con altri test.

Per risolvere questo problema applichiamo l'**Isolation Testing**. Per isolare un'unità, abbiamo due soluzioni concettuali:

1. Assumere (sperare) che tutti i moduli chiamati e le risorse siano corretti.
2. Sostituire i moduli chiamati e le risorse con versioni fittizie e semplificate, la cui correttezza è imposta per costruzione.

In ingegneria del software adottiamo la seconda strada. Per farlo, utilizziamo dei componenti specifici: i Driver e gli Stub.

- **Driver:** È il modulo che "*chiama*". Sostituisce completamente il modulo chiamante reale, impostando i valori necessari, costruendo gli oggetti (nella programmazione a oggetti) e invocando il metodo dell'unità che vogliamo testare.
- **Stub:** È il modulo "*chiamato*". È una funzione fittizia che emula il comportamento della funzione reale richiesta dal test case. L'uso degli stub ci permette di testare un modulo anche se i moduli da cui dipende non sono ancora stati implementati.

14.7.1 Test Doubles e Mock Objects

Se lavoriamo in linguaggi orientati agli oggetti (OOP), gli stub mostrano presto dei limiti: non gestiscono bene lo stato interno (gli attributi) di una classe tra una chiamata e l'altra e ci costringono a scrivere e rinominare continuamente codice fittizio per ogni singolo test case.

Per questo motivo, l'evoluzione dell'Isolation Testing prevede l'uso dei **Test Doubles** (Controfigure) e, nello specifico, dei **Mock Objects**.

Un Test Double è un'implementazione alternativa e sostitutiva di un'interfaccia o di una classe che non è stato possibile utilizzare per diversi motivi:

- Troppo lento.
- Non (ancora) disponibile.
- Dipende da qualcosa che non è (ancora) disponibile.
- Troppo difficile da istanziare e configurare ai fini del test.
- Comporta operazioni irreversibili.

- Interagisce con alcune risorse imprevedibili (ad esempio generatori di numeri casuali o connessioni di rete).

È possibile ottenere un'emulazione accurata del comportamento delle classi non ancora implementate utilizzando i Mock Objects. Un Mock Object è una simulazione di un oggetto reale. Implementa l'interfaccia dell'oggetto che deve essere simulato e presenta un proprio comportamento. Non solo fornisce risposte fisse (come uno stub), ma è anche in grado di verificare se l'oggetto sotto test lo sta utilizzando correttamente.

Per far funzionare questa architettura, è fondamentale il concetto di **Dependency Injection** (Iniezione delle Dipendenze): iniettiamo il nostro Mock Object all'interno del modulo da testare al posto dell'oggetto reale, permettendoci di eseguire il test in totale isolamento.

I Mock Objects sono però alquanto ingombranti in quanto per ogni caso di test dovremmo creare diverse classi mock e, all'interno del caso di test, istanze di oggetti mock. Questa soluzione porterebbe il codice a essere pieno di classi difficili da mantenere.

Vogliamo una soluzione che crei classi mock in modo anonimo e in fase di esecuzione, senza dover scrivere esplicitamente tutto il codice della classe. Per farlo, possiamo utilizzare **Mockito**, un framework di java apposito per il mocking.

14.7.2 Integration Testing

Una volta che abbiamo testato tutte le singole unità in isolamento, sorge una domanda: funzionano correttamente anche quando sono collegate tra loro? È qui che entra in gioco l'**Integration Testing**. Questo tipo di testing ci permette di verificare il sistema pezzo per pezzo, sostituendo gradualmente le nostre "controfigure" (stub o mock) con i moduli reali.

Se ad esempio abbiamo un modulo A che chiama il modulo B, e sia A che B hanno superato i propri Unit Test in isolamento, non c'è alcuna garanzia che l'insieme di A e B funzioni. Potrebbero esserci problemi legati, ad esempio, a combinazioni di input passati da A a B che non erano state previste da chi ha sviluppato e testato B isolatamente.

Per eseguire il test di integrazione tra A e B, in pratica, non facciamo altro che prendere i test originariamente progettati per A e sostituire al loro interno lo stub (o il mock) di B con il modulo **B** reale.

L'integrazione di un intero sistema non si fa a caso, ma segue delle strategie ben precise:

- **Top-down (Dall'alto verso il basso)**: Si parte dai moduli di livello più alto (es. l'interfaccia utente o il main). Si testano usando degli stub per i moduli sottostanti. Man mano che si scende, gli stub vengono sostituiti dai moduli reali.
- **Bottom-up (Dal basso verso l'alto)**: Si parte dai moduli terminali (quelli che non chiamano nessun altro). Si testano usando dei Driver. Man mano che si sale nella gerarchia, si usano i moduli testati per comporre i livelli superiori.
- **Sandwich testing**: Un approccio ibrido che combina Top-down e Bottom-up, incontrandosi nei livelli funzionali intermedi.

In sintesi, il flusso di lavoro ideale come ingegnere del software sarà: creare l'unità, isolarla dal resto del mondo simulando le sue dipendenze (Isolation Testing con Mock/Stub), assicurarsi che la sua logica interna sia perfetta, e infine ricollegarla gradualmente al resto del sistema reale per assicurarsi che la comunicazione tra le parti sia impeccabile (Integration Testing).

Nota: un avanzato support per le assertion è Hamcrest. Non lo metto negli appunti, ma è possibile vedere un'infarinatura nelle slide.

CAPITOLO 15

Modelli di Processo

Prima di parlare di modelli, dobbiamo definire cos'è un processo. In termini generali, un **processo** è un metodo per fare qualcosa, costituito da una sequenza di passi, attività, vincoli e risorse. Nel nostro campo, come ci ricorda lo Standard IEEE 610.12-1990, il processo di sviluppo software è l'insieme di passaggi attraverso i quali i bisogni dell'utente vengono tradotti in un prodotto software. Questo include la definizione dei requisiti, la loro trasformazione in progetto, l'implementazione del codice, il testing e l'installazione operativa.

Sviluppare software, però, porta con sé dei problemi intrinseci non da poco. È un'attività altamente intellettuale e creativa, basata sul giudizio umano, ed è pertanto difficile da automatizzare. I requisiti sono spesso complessi, ambigui e altamente volatili: il cliente stesso, all'inizio, non sa bene cosa il software debba fare davvero. Inoltre, i continui cambiamenti tecnologici e organizzativi rendono le modifiche frequenti molto difficili da gestire e i costi ardui da stimare.

15.1 MODELLI DI PROCESSO

Per gestire questo caos intrinseco, introduciamo i **Modelli di Processo**. Cosa sono? Sono descrizioni precise e formalizzate di come organizziamo la sequenza di passi dello sviluppo per massimizzare certe caratteristiche. Servono a introdurre stabilità, controllo e organizzazione, strutturando il lavoro in fasi ben definite, stabilendo i documenti (deliverables) da rilasciare al committente e fornendo linee guida per pianificare e gestire budget e personale.

Esistono diverse tipologie di modelli sviluppati negli ultimi 50 anni, tra cui il modello a Cascata, i modelli Evolutivi, Trasformazionali e quelli Agili (come XP e SCRUM). Come valutiamo se un modello è "buono" per il nostro progetto? Utilizziamo dei parametri specifici:

- **Comprensibilità**: è facile da comprendere?
- **Visibilità**: fa capire facilmente a che punto dello sviluppo siamo giunti?
- **Supportabilità**: esistono tool (CASE tools) per supportarlo?
- **Accettabilità**: le persone coinvolte manifestano il loro consenso nell'usarlo?
- **Affidabilità**: facilita l'individuazione di errori e produce software di alta qualità?
- **Robustezza**: gestisce bene i cambiamenti in corso d'opera?
- **Manutenibilità**: si adatta ai cambiamenti dell'organizzazione che lo usa?

- **Rapidità:** porta a uno sviluppo veloce?

Nota a margine: il modello più semplice, ma decisamente il peggiore da applicare, è l'approccio "Edit-Compile-Test" (o "code and fix"): non ha sovrastrutture, non incoraggia la documentazione, non permette di controllare lo stato di avanzamento, non scala su grandi progetti ed è ingestibile durante la manutenzione.

15.2 MODELLO A CASCATA - WATERFALL

Il modello Waterfall, popolare fin dagli anni '70 come reazione al caos del "code and fix", è l'applicazione all'informatica dell'approccio Tayloristico. Si basa su una rigida gestione gerarchica, passi fissi e la separazione netta delle postazioni di lavoro, in cui il processo è orientato al prodotto e non al cliente.

È un **modello sequenziale lineare**: si procede in cascata, di fase in fase senza tornare indietro (nessun riciclo), al fine di controllare meglio tempi e costi. L'overlap (sovrapposizione) tra le fasi è nullo o minimo.

Ogni fase raccoglie un insieme di attività omogenee per metodi, tecnologie, skill del personale, etc. Ogni fase è caratterizzata da:

- Attività (**tasks**).
- Prodotti di tali attività (**deliverables**).
- Controlli di qualità/stato (**quality control measures**).

La fine di ogni fase è un punto rilevante del processo (**milestone**) e i semilavorati output di una fase sono input alla fase successiva. I prodotti di una fase vengono "congelati", ovvero non sono più modificabili se non innescando un processo formale e sistematico di modifica.

I pro sono:

- Ha definito molti concetti utili (semilavorati, fasi ecc.).
- Ha rappresentato un punto di partenza importante per lo studio dei processi SW.
- Facilmente comprensibile e applicabile.
- È un modello molto rigoroso: si applica bene in qualunque contesto in cui i requisiti siano stabili e chiari.

I contro sono:

- Interazione con l'utente solo all'inizio e alla fine:
 - Requisiti congelati alla fine della fase di analisi.
 - Requisiti utente spesso imprecisi: "l'utente sa quello che vuole solo quando lo vede".
 - Errori nei requisiti scoperti solo alla fine del processo.
- Il nuovo sistema software diventa installabile solo quando è totalmente finito.

Nella realtà l'applicazione evolve durante tutte le fasi in quanto i requisiti sono volatili e gli overlap e loop sono inevitabili.

15.3 I MODELLI AGILI

Proprio per rispondere ai fallimenti degli approcci tradizionali e ingegneristici, che non riescono a gestire la mancanza di dettagli iniziali e i cambiamenti in corso d'opera, nascono le **Metodologie Agili**.

La soluzione proposta è semplice ma rivoluzionaria: **sviluppo incrementale in piccoli passi**, coinvolgendo il cliente il più possibile per minimizzare i rischi di fallimento. Questo si ottiene lavorando per **Iterazioni**. Ogni iterazione è una sorta di progetto a sé stante (che include analisi, pianificazione, implementazione, test e documentazione) con l'obiettivo di fornire al cliente software funzionante il prima possibile per ottenere feedback continui.

Nel 2001, 17 esperti di metodi software (tra cui i creatori di SCRUM e Extreme Programming) si riunirono per redigere l'**Agile Manifesto**. I valori fondamentali dell'agilità ribaltano l'approccio tradizionale e stabiliscono che si valuta:

- Gli individui e le interazioni più dei processi e degli strumenti.
- Il software funzionante più della documentazione esaustiva.
- La collaborazione col cliente più della negoziazione dei contratti.
- Rispondere al cambiamento più che seguire un piano prestabilito.

15.4 MODELLO AGILE SCRUM

Entriamo ora nel dettaglio di **SCRUM**, un framework agile pensato per consegnare il massimo valore di business nel minor tempo possibile. In Scrum, lo sviluppo del prodotto avanza attraverso una serie di iterazioni chiamate **Sprint**, che durano tipicamente dalle 2 alle 4 settimane (massimo un mese). Durante uno sprint, il prodotto viene progettato, codificato e testato. Una regola d'oro è che *nessuna modifica è permessa durante uno sprint in corso*.

Il framework Scrum si basa su tre pilastri: Ruoli, Cerimonie e Artefatti.

15.4.1 I Ruoli

- **Product Owner:** È colui che definisce le funzionalità del prodotto, ne decide la priorità in base al valore di mercato (ROI) e sceglie le date di rilascio. È lui che accetta o rifiuta il lavoro del team.
- **Scrum Master:** Non è il capo progetto, ma rappresenta il management. Assicura che le pratiche e i valori Scrum vengano seguiti, rimuove gli ostacoli (impediments) che bloccano la produttività del team e protegge il gruppo dalle interferenze esterne.
- **Il Team:** Un gruppo tipicamente formato da 5 a 9 persone, cross-funzionale (include programmatori, tester, designer) e con membri a tempo pieno. La caratteristica chiave è che il team è auto-organizzato e, idealmente, non ci sono titoli gerarchici.

15.4.2 Gli Artefatti

Il primo è il **Product Backlog**, ovvero la lista di tutti i requisiti desiderati per il progetto, definita e prioritizzata dal Product Owner. I requisiti sono generalmente espressi come **User Stories**.

Le User Stories sono descrizioni brevi per catturare il comportamento desiderato, nel formato: "*Come, voglio, affinché*".



Devono sempre includere i **Criteri di Accettazione** (Acceptance Criteria), ovvero le condizioni (pass/fail) che il software deve soddisfare per essere accettato. Un trucco per ricordarle sono le "3 C":

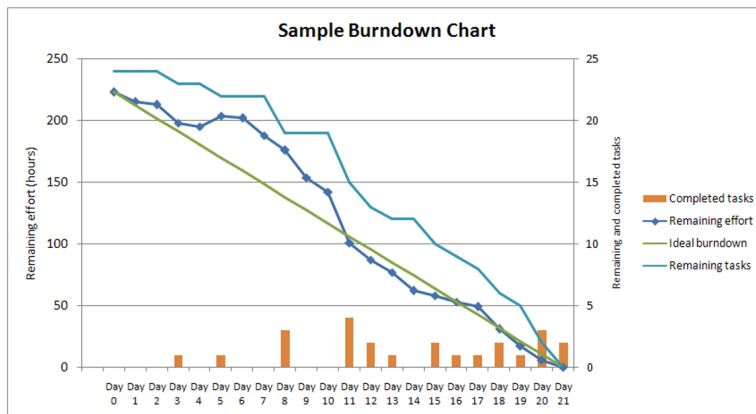
- **Card**: La scheda fisica che forza la brevità.
- **Conversation**: Il dialogo necessario per dettagliare i requisiti.
- **confirmation**: I criteri di accettazione sul retro.

Title:	Priority:	Estimate:
As a <type of user> I want to <perform some task> so that I can <achieve some goal>		
Acceptance criteria Given <some context> When <some action is carried out> Then <a set of observable outcomes should occur>		

Il secondo è lo **Sprint Backlog**, ovvero la lista dei task che il team si impegna a completare durante lo sprint corrente. Attenzione: i task (stimati solitamente tra 1 e 16 ore) vengono *scelti* dai membri del team, non vengono assegnati d'autorità.

Tasks	Mon	Tues	Wed	Thur	Fri
Code the user interface	8	4	8		
Code the middle tier	16	12	10	4	
Test the middle tier	8	16	16	11	8
Write online help	12				
Write the foo class	8	8	8	8	8
Add error logging			8	4	

L'ultimo è il **Burndown Chart**, un grafico essenziale che mostra il lavoro stimato rimanente, aggiornato quotidianamente. Permette di vedere chiaramente se il team sta smaltendo i task in tempo per la fine dello sprint.



15.4.3 Le Cerimonie (o Eventi)

1. **Sprint Planning Meeting:** Si tiene all'inizio dello sprint e si divide in due fasi. Prima c'è la prioritizzazione, in cui si valuta il Product Backlog e si seleziona lo Sprint Goal. Poi c'è la pianificazione vera e propria, in cui il team decide come raggiungere l'obiettivo, scompone le user stories in task e ne stima le ore per creare lo Sprint Backlog.
2. **Daily Scrum Meeting:** Un incontro giornaliero (Stand-up) di massimo 15 minuti. Non serve per risolvere problemi, ma per prendersi impegni davanti ai colleghi. Ognuno risponde a 3 precise domande: *Cosa ho fatto ieri?* *Cosa farò oggi?* *C'è qualcosa che mi blocca?*
3. **Sprint Review:** Alla fine dello sprint, il team presenta il lavoro completato (senza usare slide, ma tramite una demo del software funzionante). È una riunione informale a cui può partecipare tutto il mondo aziendale.
4. **Sprint Retrospective:** Incontro conclusivo di 15-30 minuti dedicato al miglioramento del processo stesso. Si analizza cosa ha funzionato e cosa no. Un format comune prevede che il team discuta cosa *Iniziare* a fare (Start), cosa *Smettere* di fare (Stop) e cosa *Continuare* a fare (Continue) nello sprint successivo.

CAPITOLO 16

Human-Computer Interaction Guidelines

Queste regole sono il frutto di decenni di esperienza e servono per guidare la progettazione delle interfacce utente (UI) e per valutarle scovando problemi di usabilità. Queste però non hanno la pretesa di essere universali o complete al 100%: vanno sempre adattate e validate per il dominio specifico per cui si sta progettando.

In letteratura molti autori hanno stilato le proprie regole, ma ci concentreremo sull'unione dei due set di principi più famosi in assoluto: le **Otto Regole d'Oro di Ben Shneiderman** e le **Dieci Euristiche di Usabilità di Nielsen e Molich**.

Le prime 8 sono:

1. Puntare alla coerenza.
2. Puntare all'usabilità universale.
3. Formire feedback informativo.
4. Progettare finestre di dialogo che consentano la chiusura.
5. Prevenire gli errori.
6. Consentire di annullare facilmente le azioni.
7. Lasciare il controllo agli utenti.
8. Ridurre il carico sulla memoria a breve termine.

Le altre 10 sono:

1. Dialogo semplice e naturale.
2. Usare il linguaggio dell'utente.
3. Ridurre al minimo il carico di memoria per l'utente.
4. Coerenza.
5. Feedback.
6. Uscite chiaramente indicate.

7. Scorciatoie.
8. Messaggi di errore chiari.
9. Prevenire gli errori.
10. Guida e documentazione.

16.1 I VARI PUNTI

16.1.1 Dialogo semplice e naturale

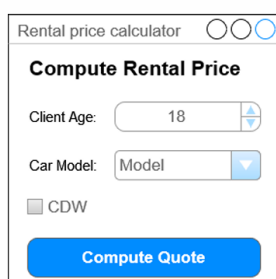
La prima regola d'oro è che l'**interfaccia deve essere il più semplice possibile** (ma non di più).

Bisogna ricordarsi che "less is more": ogni funzionalità o informazione aggiuntiva è una cosa in più che l'utente deve imparare o in cui deve cercare. Troppe informazioni sommergono gli utenti inesperti (è la Legge di Hick). Inoltre, l'interfaccia dovrebbe mappare in modo naturale il compito dell'utente nel mondo reale usando metafore, proprio come l'app bussola dello smartphone riproduce visivamente una vera bussola.

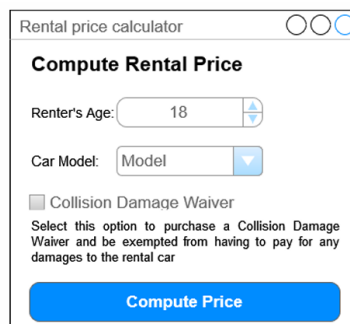
16.1.2 Parlare il linguaggio dell'utente

Il sistema deve comunicare usando parole, frasi e concetti familiari all'utente, abbandonando il gergo tecnico orientato alla macchina. Questo non vale solo per il testo, ma anche per le icone.

Se ci rivolgiamo al grande pubblico, è meglio usare termini standard che tutti comprendono; se invece progettiamo per un gruppo di specialisti, allora è corretto usare la loro terminologia tecnica.



The screenshot shows a window titled "Rental price calculator" with a standard macOS-style title bar. The content area is titled "Compute Rental Price". It contains three input fields: "Client Age:" with a text box containing "18", "Car Model:" with a dropdown menu showing "Model", and a checkbox labeled "CDW". At the bottom is a blue button labeled "Compute Quote". A red thumbs down icon is positioned below the window.



The screenshot shows a window titled "Rental price calculator" with a standard macOS-style title bar. The content area is titled "Compute Rental Price". It contains two input fields: "Renter's Age:" with a text box containing "18" and "Car Model:" with a dropdown menu showing "Model". Below these is a checkbox labeled "Collision Damage Waiver" with explanatory text: "Select this option to purchase a Collision Damage Waiver and be exempted from having to pay for any damages to the rental car". At the bottom is a blue button labeled "Compute Price". A green thumbs up icon is positioned below the window.

16.1.3 Ridurre il carico di memoria

Il nostro cervello non è bravo a ricordare dati precisi, a differenza dei computer. L'interfaccia deve farsi carico di questo sforzo. Il principio base è che il **riconoscimento è sempre meglio del ricordo**.

Se si chiede di inserire la sigla di una provincia, non bisogna costringere l'utente a ricordarsela a memoria (es. Lecce o Lecco?), ma offrire un menu a tendina da cui selezionarla. Allo stesso modo, non si può chiedere a un utente di ricordarsi a mente un lungo codice di conferma di 18 cifre per passarlo da una schermata all'altra.

16.1.4 Coerenza (Consistency)

Per essere usabile, un sistema deve avere due tipi di coerenza:

- **Coerenza Interna:** All'interno dello stesso software, situazioni simili devono richiedere sequenze di azioni simili. Le informazioni e i bottoni devono stare nello stesso posto.
- **Coerenza Esterna:** Il sistema deve rispettare le convenzioni ormai consolidate nel mondo digitale (ad esempio, le barre di ricerca o l'icona del carrello negli e-commerce si trovano quasi sempre in alto a destra).

16.1.5 Feedback e Percezione del Lavoro

Il peggior feedback possibile è *nessun* feedback. Il sistema deve costantemente **informare l'utente su cosa sta facendo**, che si tratti di successi o di errori, senza usare messaggi troppo astratti. È essenziale gestire bene i tempi di risposta:

- Sotto 0,1 secondi, l'azione sembra istantanea.
- Sotto 1 secondo, il flusso di pensieri non si interrompe.
- Sopra i 10 secondi, l'utente perde l'attenzione, per cui è obbligatorio mostrare una barra di avanzamento e una stima del tempo residuo.

Qui entra in gioco un concetto psicologico affascinante: il **Labor Perception Bias**. Le persone danno più valore a un servizio quando "percepiscono" il lavoro che c'è dietro. Se un'operazione molto complessa (come cercare il volo più economico sul web) restituisce il risultato in un centesimo di secondo, l'utente diventerà sospettoso. Per questo motivo, i designer spesso inseriscono delle finte schermate di caricamento (chiamate *labor screens*) in cui il sistema mostra i vari passaggi fittizi che sta compiendo, in modo da rassicurare l'utente.

16.1.6 Controllo dell'Utente e Uscite Chiare

Gli utenti faranno sempre degli errori, e il sistema deve offrire loro una facile via d'uscita per farli sentire in controllo. Se un'operazione dura più di 10 secondi, ci deve essere un tasto "Annulla".

Per quanto riguarda le operazioni con effetti collaterali (come cancellare dei contatti), è **preferibile fornire un tasto "Undo" (Annulla azione) piuttosto che le classiche finestre di dialogo "Sei sicuro?"**. Spesso infatti gli utenti cliccano "sì" per abitudine senza nemmeno leggere.

16.1.7 Scorciatoie

Mentre un principiante deve usare l'interfaccia seguendo poche semplici regole, gli utenti esperti devono poter **velocizzare le loro azioni tramite scorciatoie** (tasti funzione, combinazioni di tasti, doppio clic o scorciatoie nei menu contestuali). Attenzione però: se un utente deve leggere il manuale ogni volta per ricordarsi una combinazione di tasti, allora quella non è una scorciatoia valida, deve essere facile da memorizzare.

16.1.8 Messaggi di Errore

Sono il momento in cui l'utente è in difficoltà, ma sono un'ottima occasione per fargli capire meglio il sistema. Secondo Shneiderman, i messaggi di errore devono seguire quattro regole d'oro:

1. **Linguaggio chiaro:** niente codici oscuri.
2. **Precisione:** non usare frasi vaghe.
3. **Costruttivi:** devono aiutare l'utente a risolvere il problema.
4. **Educati:** non devono mai intimidire o incolpare l'utente (evita termini aggressivi come "AZIONE ILLEGALE" o "FATAL ISSUE"). L'errore alla fine è colpa del sistema che non l'ha prevenuto!. È inoltre un'ottima pratica prevedere **errori a più livelli**: un messaggio semplice per l'utente e un dettaglio tecnico espandibile (es. lo stack trace) per lo staff che fa troubleshooting.

16.1.9 Prevenzione degli errori (Slips e Mistakes)

Ancora meglio di avere ottimi messaggi di errore, è **evitare che l'utente sbagli in partenza**, per esempio facendogli usare un selettore per le date invece di fargli scrivere il testo libero. Don Norman divide gli errori in due macro categorie:

- **Slips (Sviste):** l'utente ha l'obiettivo corretto ma sbaglia ad eseguire l'azione (es. preme Invio invece di Backspace). Spesso accade per abitudine, quando segue inconsciamente un percorso frequente. Il tasto "Undo" è la soluzione perfetta per questi.
- **Mistakes (Sbagli):** l'utente esegue perfettamente l'azione che voleva fare, ma ha proprio sbagliato obiettivo a causa di un modello mentale errato (es. cancella un'intera categoria di prodotti pensando che così si cancelleranno anche i singoli prodotti al suo interno). Sono pericolosissimi perché l'utente difficilmente se ne accorgerà subito.

16.1.10 Aiuto e Documentazione

La verità fondamentale sui manuali utente è che **gli utenti non li leggono**, preferiscono tuffarsi subito nel sistema per sentirsi produttivi. Se arrivano a leggere un manuale, di solito è perché sono nel panico per un problema. Un buon sistema ideale non dovrebbe aver bisogno di documentazione, ma nella realtà è necessario averla.

Bisogna sempre ricordarsi una cosa: dire "È tutto spiegato nel manuale!" non giustifica mai un'interfaccia progettata male.

16.1.11 Chiusura e Usabilità Universale

Infine, le azioni devono avere un inizio, una parte centrale e una fine in modo da restituire un senso di chiusura (ad esempio i chiari step per il pagamento negli e-commerce). In generale, il design deve mirare alla **Universal Usability**: tenere conto delle differenze tra esperti e novizi, delle fasce d'età e delle disabilità.

Non bisogna credere che progettare per tutti renda il prodotto meno efficace. Esiste infatti il "*Curb cut effect*" (l'effetto degli scivoli sui marciapiedi): progettare un aiuto per una categoria specifica (come le rampe per le sedie a rotelle) finisce per rendere la vita più comoda a tutti, che abbiano un passeggino, una valigia o un carrello.